

Phase-7 Engineering Notes

A Running Log of Observations, Ideas, and Notes for the Phase-7 Project

Status: **Living document** — updated as new entries are added

Started: **August 25, 2026**

Origin: **Began as a single standalone note (credentials gap + pacing observation); converted into this standing log at Viktor's request, so future notes and ideas get appended here instead of each needing a new document.**

Relationship to the Constitution: **entries here are never automatically part of it.** The frozen 21 / 7 / 10 / 6 scope (ratified August 26, 2026) only changes through the Constitution's own process — see “How This Document Works.”

“Evidence gets the final vote, on both sides of that relationship” — including when the evidence is a note neither of us would have thought to write down on its own.

Contents

How This Document Works

Entry Log

Document History

How This Document Works

A place for anything worth writing down that doesn't belong in the Constitution and isn't substantial enough — yet — to deserve its own standalone PDF. New entries get appended at the end, in order, as they come up.

Every entry gets the same shape: a number, a date, a short title, a status tag, and a body. Entries are never rewritten after the fact — if something changes or gets resolved, that becomes a new entry that references the old one by number, the same no-silent-edits discipline the Constitution holds itself to.

Status tag	What it means
CANDIDATE — NOT ADOPTED	A possible future addition to the Constitution. Logged for the record, not adopted — the scope freeze means it waits for the audit, the same as every other future candidate.
OBSERVATION — FILED FOR THE RECORD	A reflection, insight, or pacing note. Not asking for any action — just worth having written down rather than left in chat history.
REFERENCE — FILED FOR THE RECORD	Practical, factual information worth keeping — a file list, a readiness confirmation, something a future entry or the audit might need to point back to.
RESOLVED — SEE ENTRY #N	Marks an entry whose open question has since been settled elsewhere, pointing to the entry that resolved it, rather than editing the original.
IDEA — UNEVALUATED	A raw feature idea for the engine itself. Not a Constitution candidate, not an observation about the project, not a settled fact — just a concept written down before it has been scoped, designed, or checked against the Constitution.
DECISION — ADOPTED	A decision that has actually been made and acted on, usually closing one or more earlier entries. Records what was decided, by whom, and what it changed — including any judgment call Claude made rather than Viktor, so it can be reversed knowingly rather than discovered later.
★ Gold-framed entries	Not a status tag — a small number of entries Viktor has asked to be marked especially important get a gold frame and a kicker line instead of, or on top of, their normal status tag. Rare on purpose. If it stops being rare, it stops meaning anything.

None of this bypasses the Constitution's own process. A CANDIDATE entry becomes part of the Constitution only the way any future amendment does — proposed once the audit has run and the scope freeze is deliberately revisited, not by accumulating enough notes here.

Entry Log

In order added. Entries 1 through 3 below were the original standalone Engineering Note #1, split into individual entries when this became a standing log.

#1

Credentials & Secrets Handling Gap

August 25, 2026

CANDIDATE — NOT ADOPTED

“API keys, exchange access tokens, and any other credentials the engine or its operator relies on must never be logged, hardcoded, committed to version control, or surfaced in decision-support output.”

Why it's being logged: Nothing in the Constitution, and nothing in any of the three outside assessments, addresses credentials handling. This project is explicitly headed toward being run by other people, potentially with their own exchange credentials in play. A leaked key is a categorically different kind of failure than a wrong prediction — it's irreversible and harms the customer directly, not just the engine's credibility. Sits close in spirit to Tier 1, Item 1 (“Analysis ≠ authority”): the engine already isn't trusted to act on markets on its own; it certainly shouldn't be trusted to mishandle the keys that would let it, or an attacker, do so. Proposed tier, whenever this is actually considered: Tier 1, alongside “Tool, Not Autonomous Actor” — though exact placement is a judgment call for the audit, not a decision made here.

#2

Project Pacing, Mid-Review

August 25, 2026

OBSERVATION — FILED FOR
THE RECORD

Said plainly, without alarm: this is a pacing observation, not a criticism. Nearly everything built in this project's governance thread so far has been governance — the original principles discussion, the Constitution's v1.0 draft, three independent outside reviews, two revisions, and a separate Documentation & Change-Log Standard. None of it has yet touched the sixteen actual files that make up the engine's codebase. That's not wasted effort — Tier 1, Item 17 exists precisely because this project moved to backtesting once before without that kind of foundation in place, and paid for it. But it does mean the real test of whether any of this holds up under real code is still ahead: the audit. Not a suggestion to rush it — just worth being aware that a great deal of the work so far has been deciding how to judge the engine, and comparatively little of it, yet, has been judging the engine.

#3

Audit Readiness Confirmation

August 25, 2026

REFERENCE — FILED FOR THE
RECORD

A practical note, not a request for a decision: when Viktor is ready to ratify and start the audit, the engine's code is already available to work from — `bias_engine.py`, `btc_context.py`, `data_fetcher.py`, `decision_model.py`, `engine_core.py`, `entry_model.py`, `exit_model.py`, `indicators.py`, `panel_render.py`, `risk_model.py`, `signal_router.py`, `structure.py`, `trend_health.py`, `live_trading.py`, and `test_live.py`. The audit can begin module by module against all four tiers, using the Minimum Viable Audit gate and the finding schema already built into the Constitution's Next Steps section, the moment Viktor says go — no preparation needed beforehand, on his own timeline.

#4

Constitution Given Equal Priority to Core Engine Code

August 25, 2026

OBSERVATION — FILED FOR
THE RECORD

"This Constitution Blueprint is exceptionally important for the project, so I am going to take my time with it. It is the foundation of the entire project, just as important as the core and main .py file."

Why it's being logged: Viktor's own statement of priority, worth keeping in his own words rather than paraphrased away. It sets real expectations for pacing — there's no pressure behind ratification, and a slow, careful read is the correct way to treat a document being placed on equal footing with the engine's own core code, not a deviation from how this should go. It also reinforces Entry #2: governance work being unhurried isn't lost time against the codebase — for a document doing this job, it's the point.

#5

Configurable Risk Tolerance (% Basis) for Entry/Target Pricing

August 25, 2026

IDEA — UNEVALUATED

"The engine should have options to change risk tolerance on a percentage basis. If I want to risk 5, 10, 15, 20, or 25 percent of my investment, I should be able to put that specific percent risk tolerance into the engine. The engine then gives the right price targets and entry price in relation to the risk tolerance."

Why it's being logged: A raw feature idea, not yet scoped or designed, so it's filed here rather than treated as a decision already made. Three things worth noting for whoever eventually designs it. First, it sits well with Tier 1, Item 14 ("Risk Is Not Conviction") — a chosen risk-tolerance percentage is a risk-layer input, separate from how confident the engine is in a direction, which is exactly the separation that principle already asks for. Second, it will need a precise, unambiguous definition before it's built — per Tier 1, Item 9 ("Every Measurement Has a Precise Definition"), "5% risk tolerance" has to be spelled out exactly: percent of what (account equity, position size, margin used), and exactly how that number changes the stop-loss placement, the entry price, and the target price, before any code is written against it. Third, per Tier 3's hypothesis-driven development process, this deserves proper scoping and design as its own small project — a stated hypothesis, a definition, a test — rather than being bolted onto the entry or exit logic ad hoc. Nothing here is a decision to build it; it's the idea, captured accurately, so it isn't lost before the audit and design work catch up to it.

#6

Audit-Time Checks Flagged by the Fourth External Review

August 26, 2026

REFERENCE — FILED FOR THE
RECORD

A fourth reviewer of the Constitution flagged four specific things the audit will need to check that go beyond what a Non-compliant/Compliant reading of the current code can show on its own — filed here so they aren't lost before the audit reaches them. **Item 1 (Tool, Not Autonomous Actor)**: currently enforced by the code simply not containing order-placement logic — one bug or one convenience import away from being false. The checkable version: does the running process have access to API credentials with trade permissions at all? If yes, Item 1 is aspirational regardless of what the code currently does — read-only keys make the invariant structural instead of behavioral. Proposed for the Minimum Viable Audit gate. **Item 5 (Reproducibility)**: recording source, version, and timestamp metadata describes an input but doesn't guarantee it can be recovered later if an exchange's API revises or rolls off historical data. Reproducibility may require archiving the raw pulls themselves — whether to do that, and for what retention window, is a decision the audit should force explicitly. **Items 9 and 10 (Precise Definitions / Consistent Semantics)**: currently unauditable, because there's no single place the definitions actually live. The proposed audit deliverable is a term registry — every term appearing in a module signature or on the panel (confidence, trend strength, momentum, alignment, risk) defined once, with its type, range, and units. That artifact would also be the permanent fix for the class of bug that broke fourteen modules once already. **Item 2 (Look-Ahead Bias), Minimum Viable Audit gate**: the document currently discusses look-ahead mostly as a backtesting failure, but backtesting is Step 9 — the surface it usually hits doesn't exist yet, which risks a hollow “Compliant” at the very first gate. In a live engine, look-ahead is real but specific: indicators computed on the current unclosed candle and treated as closed, centered rolling windows, resampling that borrows from the following bar, and any signal recomputed on data revised after the decision timestamp. Naming those explicitly is what would make that gate check something real.

#7

Optional: Split Reviewer Commentary Into an Annex

August 26, 2026

CANDIDATE — NOT ADOPTED

The fourth reviewer noted, as an optional structural observation, that roughly five pages of the Constitution are rules and twelve are process journal, glossary, and reviewer commentary — and that these have very different lifespans: the rules get checked against code for years, the review log is stale the moment the next revision lands. Their suggestion, if the front matter ever gets in the way: split the change log and reviewer impressions into a separate annex document, keeping the glossary where it is since it earns its place. Not acted on in Revision 3 — logged here as a candidate restructuring of the document itself, not a rule change, for Viktor to decide on later if the Improvements section keeps growing with each revision.

Viktor asked what philosophical questions he should be asking himself when designing and building this engine, and brought back a ten-question input worth taking seriously. Ran each question through the input's own stated filter — a question earns its place only if answering it differently would produce a different engine — rather than accepting or logging all ten uncritically. Five passed and are now Future Amendment Candidates in Constitution Rev 4: a written kill condition, a confidence calibration log, an edge-persistence hypothesis requirement, UI-level enforcement of Item 1, and a two-analyst lens-vs-property test for Item 9. Two more didn't need new rules, only better rationale text for rules already there — the Duhem-Quine problem sharpening Tier 3's Controlled Changes, and ergodicity sharpening Tier 1, Item 14 (this second one connects directly to Entry #5 above: it's the mathematical reason sizing isn't a bolt-on feature). The remaining three — what the project is actually optimizing for, the returns-vs-understanding trade-off, and a project-level kill condition — are about Viktor and the project rather than the engine, and correctly didn't pass the filter. They're addressed instead in a new standalone document, `Phase7_Tier0_Companion.pdf`, written specifically to hold philosophical material that sits above the engine rather than inside it. Source not yet attributed in either document.

“At this point i think it is important to yet again explain very directly, the size of the bet or money put on the table in a trade is of zero importance. The engine's purpose is to produce a result that can support the trader to establish the correct entry price, the correct T1 target, the correct T2 target and the correct T3 target and also establish the safety and quality of each individual trade. Not speculating in how much money is put on the specific trade. I've think we already gone over this. Thoughts? — with a follow-up clarification: 'Or should be put into a trade.' And, once the correction below was already underway: 'The engine will have a setting where you can adjust risk tolerance of your entire equity like we said before 5,10,15,20,25%. But ultimately it is up to the trader to decide how much money he wants to put on the table and the risk that comes with that.'”

Why it's being logged: This is a direct correction of scope drift I (Claude) introduced, and it's worth naming plainly rather than quietly fixing. Constitution Revision 4's ergodicity rationale under Item 14, its own revision_box summary, and the Tier0 Companion's Question 6 all drifted into language implying the engine should have a hand in position sizing or money-amount recommendations — one line in the Companion went as far as calling sizing “the thing that decides whether the signals ever mattered in the first place.” That's wrong, and Viktor caught it. Ergodicity is a real reason entry, stop, and target *prices* deserve as much engineering care as the directional read — it was never a reason for the engine to touch money amounts. The engine's entire output is: the correct entry price, the correct T1, T2, and T3 targets, and an assessment of the safety and quality of the trade. How many dollars, or what fraction of an account, a trader actually puts at risk is the trader's decision alone, in every direction — how much money is put on the table and how much should be put into a trade — per Tier 1, Item 1. This does not touch Entry #5 above: the configurable risk-tolerance percentage (5/10/15/20/25% of equity) is still exactly what was asked for there — an input the engine uses to shape where entry, stop, and target *prices* sit. It was never a request for the engine to size positions in dollar terms, and it isn't being treated as one now. Fixed at the source in Constitution Revision 5 (Item 14 rationale, its Revision 4 summary left as historical record, plus the glossary's ergodicity definition) and Tier0 Companion v1.1 (Question 6). Entry #8 above is left unedited per this document's own no-silent-edits rule — its “sizing isn't a bolt-on feature” phrase carries the same overreach and should be read in light of this entry, not corrected in place.

"We absolutely need to make rules and safety protocols for the API handling. No doubt. [...] In truth neither of us needs to be the auditor, we can outsource that to a A.I with the most powerful model. It might cost us a little bit of dollars each time but i think it is worth it."

Why it's being logged: Two entries in this log stop being open items today. Entry #1 proposed credential handling as a candidate Tier 1 invariant and noted it wasn't covered by the Constitution or any of the three outside assessments; Entry #6 recorded the fourth reviewer's sharper version — that Item 1 was enforced only by the code happening not to contain order-placement logic, and that read-only keys would make it structural. Both are now adopted. Constitution Revision 6 adds Tier 1 Items 18 through 21 (categorical read-only market access; withdrawal permissions never enabled, kept separate as a floor that survives any future amendment to Item 18; credentials never exposed; operator credentials stay with the operator), and Item 18 joins Items 2, 3, and 6 in the Minimum Viable Audit gate. Operational detail lives in a new companion document, `Phase7_Credential_Security_Protocol.pdf`, deliberately kept out of the Constitution so that document stays a register of invariants rather than an operations manual — and so the protocol can be improved without touching a ratified rule. Two judgment calls in there were mine, made at Viktor's invitation and both reversible: read-only is stated categorically rather than as a default, because "by default" reintroduces exactly the undocumented escape hatch the fourth reviewer found in Items 4 and 12 — if an execution capability is ever wanted, it should cost a constitutional amendment, not a config flag. And operator credentials became a fourth invariant rather than protocol-only detail, because it passes the Tier 0 relevance test: it constrains architecture (no upload path, no central key store, no telemetry capable of carrying a key), and a leak there harms someone who trusted the engine rather than the person who built it. **On the auditor:** Revision 6 also names an independent auditor as the party responsible for Steps 3, 4, and 8, closing the gap the fourth review opened and this document's Entry #2 pointed at from another direction. Claude prepares the package and writes zero findings; the external auditor produces them; Claude answers each one adversarially, including against its own work; Viktor adjudicates. Worth recording honestly: Viktor's framing that a second model is "another reviewer, not some magical oracle" is now written into the Constitution as the distinction between authorship independence and judgment independence — the auditor has no stake in the engine looking good, but shares training data and reasoning habits with the assistant it checks, so a clean audit is evidence, not proof. That is the same correction Revision 3 already had to make about four reviewers agreeing, applied before the mistake gets made a second time rather than after.

"I am considering implementing Grok 4.7 to be the sole auditor when it releases. In fact i know i will. It is decided."

Why it's being logged: This closes the last open question the Constitution's independent-auditor requirement (Rev 6) left blank: which model actually fills the role. Recorded in Constitution Rev 8 as the current operational plan, deliberately kept out of the frozen requirement itself — the requirement stays general (no stake in the outcome, no shared authorship with the builder) so that if Grok isn't ready or doesn't hold up when the time comes, swapping the choice doesn't require a constitutional amendment, just an updated plan. Worth naming the good timing: Rev 7, written earlier the same day, had just warned that a fresh Claude session (Reviewer 4's actual identity) gives real authorship independence but weak judgment independence, since it shares lineage with whatever it's checking. Grok answers that directly — different company, different training data, no shared lineage with Claude at all. That's the harder of the two things the auditor needs, genuinely satisfied. The no-stake-in-the-outcome half still has to hold regardless of which model ends up in the seat; picking a different vendor doesn't get to skip that. One real dependency, stated plainly rather than glossed over: Grok 4.7 has not released as of this writing, so Step 3 of the audit sequence can't start until it does and Viktor judges it capable of reading sixteen modules of code and a 44-rule register competently. That doesn't block ratification, and it doesn't block Step 2a — the audit package can be assembled now, on Claude's own timeline, so it's sitting ready the moment the auditor exists.

#12

The Audit Loop, Traced — and Where the Weight Actually Sits

August 26, 2026

KEY INSIGHT — LOOP
INTEGRITY

"With Grok 4.7 we have god damn near created the perfect loop for this project!"

Not a new decision — a reflection on the structure Revisions 6 through 8 built, worth keeping visible rather than letting it dissolve back into the mechanics it describes. Viktor asked for it to be marked, and it's marked.

The loop, traced: Claude builds the engine and prepares the audit package, but writes zero findings — the builder cannot self-certify. Grok evaluates the engine against the full 44-rule register and produces the findings, but holds no authority to declare anything final. Claude answers every finding adversarially, including against its own work, so nothing gets buried by silence. Viktor adjudicates every disagreement, so the loop never resolves itself inside a machine — it always terminates in a human decision. Fix, regression test, re-audit. No party in that chain can mark its own homework, and no party holds unilateral authority either. That is the actual structure eight revisions, four outside reviews, and two self-corrections were sanding toward.

Two things keep it short of perfect, named here rather than left implicit, because that has been the discipline of this whole project. First: the loop is only as sound as the package Claude assembles for Grok. Claude writes no findings, but does choose what evidence goes in — the code, the failure history, which "already known" items get flagged as open questions. That's real leverage held by a party with zero votes on the outcome. Revision 6 addressed this partially ("the artifact and the evidence, not the argument"), but a fully adversarial package would ideally be checked by someone other than its author too — not solved today, filed honestly as a residual. Second: the loop's integrity lives entirely in how seriously Viktor engages with each adjudication. The design guarantees that a disagreement between Claude and Grok surfaces. It cannot guarantee that surfacing gets read closely rather than rubber-stamped. That isn't a defect in the document — it's an honest statement of where the weight actually sits: on the one step nothing here can automate.

Neither point is a reason to slow down ratification. They're the reason the audit findings, once Grok exists, deserve a close read rather than a count.

"Perhaps we should add this reasoning in the documents after all? Maybe in notes?"

Not a change to Entry #11 — Grok is still the named plan there, and this doesn't touch that. Viktor separately noted that if a better AI turns up before Grok 4.7 is actually in use, Grok isn't locked in — which is exactly the flexibility Constitution Rev 8 built in on purpose, so this required no document change to be true. What's worth keeping is the reasoning for whenever that comparison actually happens, since it isn't written down anywhere else. Three separate axes, easy to blur into one impression of "good model": **independence** is the actual Constitutional requirement — no stake in the outcome, no shared authorship with whatever built the engine — and it's binary, not a spectrum a stronger model can compensate for. **Power** buys a more thorough audit once independence is already satisfied, not a substitute for it. **Cost** matters more than it looks like it should, for a reason specific to this design rather than budget in general: Step 8 makes the audit a recurring expense, not a one-time one — every fix triggers a re-audit. An expensive auditor doesn't fail on the first run. It fails gradually, as the fifth or tenth re-audit gets deferred, shortened, or skipped because paying for it again is annoying. Cheap is what keeps the loop actually running instead of quietly degrading into "close enough." Whoever ends up in the role, these three should be checked in this order — independence first, as a gate, then power and cost as the actual trade-off.

"I have read the document and i APPROVE!"

Viktor reviewed Constitution Rev. 8 — the document itself, plus two companion reading aids built for this specifically: a 12-page curated excerpt pulling only the pages bearing on the three judgment calls made on his behalf (Items 18, 19, 21), and a plain-language Swedish explainer covering the same ground without technical or legal wording — then approved it. Rev. 8's own "What ratification means" text defines the act narrowly: Viktor writes "ratified," and a dated row is added to Version History, nothing heavier required. His actual words were "I APPROVE," not the literal word "ratified" — recorded here exactly as written, since the intent is unambiguous and the document's own rule is about the dated record existing, not about matching one specific word. That record now exists: a RATIFIED row in the Constitution's own Version History, dated August 26, 2026. Effective immediately: the scope freeze is in force for real, not provisionally. The 44-rule register (21 / 7 / 10 / 6) does not grow, shrink, or reword again until the audit actually runs and finds a gap — the same freeze this document has described as pending since Entry #3. Step 1 of the audit sequence is done. Step 2a — Claude assembling the audit package, writing no findings — can start now. Step 3 still waits on Grok's release, per Entry #11.

#15

Gemini's Assessment of the Ratified Constitution

August 26, 2026

EXTERNAL ASSESSMENT —
GEMINI

"If evaluated as an engineering constitution for an AI-driven trading or quantitative system, it easily ranks among the most disciplined, self-correcting frameworks ever put to paper."

Viktor asked Gemini, in a separate session, to rate the finished Constitution after ratification, and asked for the reply to be logged and highlighted — the same treatment as Entry #12. Worth noting for context, not to undercut it: this is Viktor's own solicited opinion, not a structured finding against the 44-rule register, and it isn't the audit — Grok's Step 3 review is still the one that actually counts. Gemini was also one of the three original reviewers behind Revision 1, so this is a return visit from a source already on record, now looking at the finished, ratified shape rather than an early draft. Four points from its assessment, condensed: it credited the document for actively correcting its own reasoning rather than only stating good intentions — pointing specifically to Revision 3's fix, where four reviewers agreeing with each other got correctly downgraded from "validation" to "plausibility check." It credited Item 18 for converting a promise into a structural fact — read-only market access enforced at the exchange itself, so unauthorized trades stay impossible even under a fully compromised engine, not merely against one that behaves as written. It credited the Claude/Grok split for solving the conflict of interest built into most internal audits, where the same team that wrote the code also grades it. And it credited Tier 1, Item 17 specifically as a rule written from a real, lived failure — backtesting breaking a previous build — rather than a theoretical best practice with no scar behind it. Its closing line: that the document operates on a different plane of paranoia, rigor, and self-governance than standard software engineering guidelines. Condensed here to these four points rather than reproduced word for word, but nothing in the condensing changes what it credited or why — filed here per Viktor's instruction to log and highlight it, not to soften it.

Entry #15 stands as written — this is not a correction of it, and per this document's no-silent-edits rule it is not edited. What that entry does not record is how the assessment was obtained, which bears on how much weight it carries. Viktor asked Gemini for an opinion on work Viktor had done. The party being evaluated commissioned the evaluation, and the evaluator had no stake in delivering a disappointing answer. That is the same structural problem this project has now corrected twice: Revision 3 downgraded four reviewers agreeing with each other from “validation” to a “plausibility check,” and Revision 7 separated authorship independence from judgment independence after Reviewer 4 turned out to be Claude. A solicited favourable opinion is a third instance of it, and worth naming before it becomes a habit — the gold frame on Entry #15 marks that Viktor asked for it to be highlighted, not that its epistemic status is stronger than any other entry here. Viktor then commissioned the counterweight: a deliberately harsh critical review of Gemini's follow-on career assessment, delivered as Viktor_Karriarbedomning_Kritisk_Granskning.pdf — a personal document, outside the Phase-7 archive. That review's own first page names the unresolved conflict: Claude wrote most of the documents being praised and is therefore also the wrong party to judge them. Naming the conflict does not remove it. The one assessment that will carry real weight is still Step 3 — the independent auditor, against the 44-rule register, on the actual code — and it has not happened yet.

“Step 3”

Per the ratified Constitution, Step 2a is Claude's role in the audit sequence: assemble the code, the evidence, and the failure history Step 3 will need, and write zero findings. Viktor granted access to the working repository on his machine for this purpose; Claude read all 19 Python files (16 register modules plus three root-level entry points) and four evidence files under Logs/ that are not part of the public repository (they are gitignored). The result is Phase7_Step2a_Audit_Package.pdf, plus two bundles: the 19 source files, and the four evidence files. The package includes a file manifest, the literal results of two searches run against the code for Item 18 evidence (order-execution call patterns: zero matches; credential fields: two matches, both empty strings in core/config.py), seven self-documented fixes already present as comments in the code (labeled A11 through A13 and others, none written for this package), and two items flagged as open questions for Step 3 rather than resolved here — the BTC-Adjusted AERO Prediction feature's validation status, and whether roadmap Layer 5 (entry multipliers), absent from all 19 files, is a gap or a deliberately deferred feature. One correction this package makes to the project's own record, without editing the original: Entry #3's file list is now superseded. The engine has grown since that entry was written — two modules it doesn't mention (models/btc_context.py, models/decision_model.py) now exist, and two it does mention (indicators/supertrend.py, models/bias_state_machine.py) have been removed. The audit package's Section 1 is the current, accurate file manifest; Entry #3 stays as written, a record of the architecture at the time it was logged. Step 3 remains blocked on nothing but Viktor actually running it.

#18 An Independence Check Before Step 3: Grok Had Already Seen the Constitution

August 26, 2026

OBSERVATION — FILED FOR
THE RECORD

Before Step 3 runs, Viktor mentioned that Grok had already looked at the Constitution in an earlier, separate conversation — casually, the way Gemini did before Entry #15. That raised the same structural concern named twice already in this project (Revision 3's downgrade of agreeing reviewers; Entry #16's naming of Entry #15's solicited-opinion problem): if the model that already reacted favourably to the Constitution in one conversation is the same model, with the same conversational memory, asked to formally audit the code against it in Step 3, a consistency bias toward staying favourable is a real risk, not a hypothetical one. The fix does not require touching anything published. It requires running Step 3 in a conversation with no shared memory of the earlier one, so Grok's evaluation of the code carries no carried-over goodwill toward the document it is grading the code against. Viktor confirmed the earlier conversation's history is cleared and Step 3 will run fresh. The public GitHub repository was considered as a second, weaker version of the same risk (Grok's web search could in principle read the README's confident framing before reading the raw code) and deliberately left published — the risk that mattered was the shared conversational context, not the existence of public documentation, and unpublishing working, licensed, verified material to guard against a smaller and more speculative risk was judged not worth it. Named here for the same reason Entry #16 named its own version of this: independence problems get written down, not quietly avoided.

#19 Grok's Assessment of the Constitution — an External Review, Not Step 3

August 27, 2026

EXTERNAL ASSESSMENT —
GROK

"The document has already done the hardest part: it made the rules hard to quietly bend. The next test is whether the engine can survive being measured against them without the rules themselves moving."

Viktor asked Grok (free tier) to review the ratified Constitution and shared the result here. Logged as an external assessment, in the same category as Entry #15 — not as Step 3, which is the audit of the code against the register and has not been run. Claude verified its factual claims against the document: the item numbers, names and characterisations it cites are correct throughout, including the Minimum Viable Audit gate composition and the Roles & Authority wording. It is not a review that hallucinated content and sounded confident. Its strongest section argues against its own weight: it states plainly that it shares training data and reasoning patterns with Claude, that correlated blind spots therefore remain possible, and that naming a specific vendor does not manufacture independence — the same conclusion this document already reached, arrived at independently. Its substantive criticisms: the Minimum Viable Audit gate should also cover Items 1 and 13; Items 9, 10, 11 and 16 remain hard to falsify cleanly without a formal term registry, a real dependency graph, and a concrete standard for "demonstrated value"; and Roles & Authority should be softened post-ratification so the auditor and Viktor, not Claude, hold final technical adjudication. Those are rule changes and the freeze forbids them until the audit finds a gap, so they belong in Future Amendment Candidates rather than in the document now. Two honest weaknesses in the review: it opens with praise of the kind Entry #16 warned about, though unlike that case real criticism follows it; and parts of it read this document's own conclusions back as findings. It also missed the internal contradiction recorded in Entry #21, which was findable from the document alone. Grok stated it is prepared to perform the real audit once a capable version is available — see Entry #20 for why that is no longer the plan.

Viktor stopped Step 3 before it ran and changed the plan. The trigger was cost structure: Grok's capable tier requires a monthly subscription of about thirty dollars, which is poor value for something run a few times a year, and Viktor was not willing to carry a recurring cost for intermittent use. Investigating that opened a more useful fact — pay-per-token API access through an aggregator (OpenRouter: no subscription, no minimum spend, credits that do not expire) costs roughly one dollar for a full audit run of this package, even on the most expensive frontier models. Cost is therefore not the binding constraint it appeared to be, and the auditor should be chosen on independence and capability instead. On independence the position is worse than it looks: every model that has touched this project is now compromised to some degree. Claude wrote the Constitution, most of the engine, and the audit package, and was also Reviewer 4. Gemini and ChatGPT were two of the three original reviewers, and Gemini additionally supplied the solicited praise in Entry #15. Copilot was the third and shares a model family with ChatGPT. Grok has now read and assessed the Constitution (Entry #19). That is a consequence of thoroughness, not carelessness, but it exhausts the obvious list. The new plan: Kimi K3 (Moonshot AI) as intended primary auditor — no prior involvement, a different training lineage, and independently strong on code work — with DeepSeek and GLM (Z.ai) available as further independent runs, all through one pay-per-use account. Running several is deliberate and its purpose is narrow: not to treat agreement as evidence, which Revision 3 already ruled out and which still stands, but so that disagreement between independent auditors becomes visible at all. A single auditor cannot produce that signal. No revision of the Constitution is required for any of this: Revision 8 named Grok explicitly as “the current plan, not as a fifth thing this document requires,” and Entry #13 set the substitution criteria in advance. Recorded in the Constitution as a new AUDITOR row; the Rev. 8 text naming Grok stays as written.

#21 A Contradiction Found in Claude's Own Work, Recorded Rather Than Repaired

August 27, 2026

OBSERVATION — FILED FOR
THE RECORD

While checking Entry #19's claims against the Constitution, Claude found that the document contradicts itself. Next Steps defines the Minimum Viable Audit gate as four items — 2, 3, 6 and 18 — and explains why Item 18 belongs in it. The conflict-of-interest safeguards, written in the same revision, then refer to “the Minimum Viable Audit gate items (Items 2, 3, 6),” leaving Item 18 out. The effect is not cosmetic: Item 18 is the invariant that makes Item 1 structural instead of aspirational, and it is currently the only gate item not covered by the rule that gate items get a second check from a reviewer who did not write the code. Claude wrote both passages in Revision 6. Viktor asked whether to fix it and Claude recommended against, for reasons that are worth stating rather than assuming. The party that made an error should not be the party that decides the evidence of it disappears — that argument alone settles it. A repair would also erase the fact that this document once contradicted itself, where a dated record keeps it visible permanently; that is the same reasoning behind this log's no-silent-edits rule, applied to Claude's own mistake rather than to someone else's. A defensible case exists that a repair would be permitted under the freeze — it adds no rule, changes no item, and makes the standard stricter rather than looser — but that case has the shape “this change is an improvement, therefore it is allowed,” and deciding what counts as an improvement is exactly the discretion the freeze was written to remove. The first thing that looks worth fixing after ratification is the worst possible moment to open that door. Finally, an internal contradiction is precisely what Step 3 exists to record a finding on, so resolving it belongs to the auditor. Recorded in the Constitution as a DEFECT row, with the stricter reading governing in the meantime: Item 18 is a gate item and does get the independent second check. The audit must record a formal finding on which passage stands.

#22 Run 1 — a Blind Review Before the Compliance Audit, and What It Found

August 27, 2026

AUDIT RUN 1 — RECORDED

Step 3 as written sends the auditor the engine and the register together. That design has a blind spot: an auditor holding a 44-item checklist tends to find things on the checklist. It cannot easily tell you the register is missing an invariant, because nothing prompts it to look for one. So the audit was run in two orders rather than one. Run 1 gave a model the source code alone — no Constitution, no audit package, no indication that a standard exists — and asked only what would fail. DeepSeek V4 Pro did that run, on OpenRouter, for roughly fifty cents. The ordering is not negotiable, for a mechanical reason rather than a procedural one: a model that has read the Constitution cannot un-read it for a later blind pass, so the blind run comes first or not at all. It returned ten ranked findings. Claude checked every one against the real source rather than accepting them, and all ten held. The most serious: a dead safety gate in `trend_health.py` that tests for string values the `STRUCTURE` column never contains, so it can never fire; and indicator fallbacks that invent plausible directional values on failure rather than reporting failure — a `SuperTrend` exception yields `ST_Direction 1.0`, which `bias_engine` scores as +100 at 15% weight, injecting a permanent bullish tilt precisely when the indicator is broken. One DeepSeek citation, a specific issue in a third-party library's tracker, could not be verified and was recorded as unverified rather than adopted. The blind run was Claude's idea and appears nowhere in the Constitution; whether it earns a place there is a question for after the audit, not during it.

#23

The Audit Splits Into Three Runs Because the First One Ran Out of Room to Think

August 27, 2026

DECISION — ADOPTED

The first attempt at Run 2 — the actual Step 3 compliance audit, with Kimi K3 and all five materials — produced a 16,384-token reasoning trace and then stopped without writing a single finding. The cause was banal: OpenRouter's chat interface leaves Max Tokens unset, falls back to 16,384, and bills reasoning against that same ceiling. The auditor spent its entire budget thinking and was cut off before the answer began. Two things follow, and only one of them is about settings. The settings fix was to raise the ceiling to 64,000, enable reasoning at maximum effort, and clear OpenRouter's default system prompt, which instructs models to hide content inside collapsible sections — unhelpful for a document meant to be read whole and archived. The second consequence was structural: maximum effort across all 44 items would risk the same truncation at a higher ceiling, so the register was divided across three runs, each getting the full budget for a quarter of the work. Run A takes the Minimum Viable Audit gate, Run B the remaining seventeen Tier 1 invariants, Run C Tiers 2 through 4. The division follows the Constitution's own priority ordering rather than being invented for convenience. This changed the wording of an instruction that had already been recorded verbatim, on the stated reasoning that how the auditor was asked bears on what the auditor found — so the change is recorded too, as a new Section 6 in the execution-instructions document, carrying all three scope paragraphs. Splitting an audit across three sessions is a scheduling decision forced by a token ceiling. It resolves nothing, and every one of the 44 items still needs a finding before the freeze lifts.

“Layer 5 (entry multipliers) does not appear anywhere in this package's 19 files under that name.”

That sentence is Section 5.2 of the Step 2a audit package, written by Claude and handed to the auditor as one of two open questions. The string “Roadmap Layer 5” appears three times in the source — twice in `entry_model.py`, once in `engine_core.py` — under exactly that name. Kimi K3 checked, and said so: the claim is false. It is worth being precise about what kind of error this is. Claude did not misjudge a borderline call; it asserted the absence of a string in a document whose credibility rests on presenting literal search results, without running the search. The disclosure paragraph in the auditor's instruction — telling it to verify the package's factual claims rather than inherit them — was written as a precaution and turned out to be load-bearing on the first attempt. The audit itself: Item 2 (Look-Ahead Bias) Compliant, with the swing-confirmation logic in `structure.py` singled out as correctly built, and a standing caveat that the codebase's pervasive backward-fill becomes a genuine leak the moment anything evaluates historical decision points, which is what Step 9 will do. Item 18 (Read-Only Market Access) Compliant: zero execution surface, no HTTP verb but GET, no credential presented at all. Item 3 (Data Integrity) Non-compliant, Critical — none of missing candles, duplicate candles, impossible prices, timestamp ordering, staleness or abnormal volume is detected, and defects are silently fabricated away by `ffill/bfill` instead; a macro-timeframe failure is quietly reinterpreted as NEUTRAL. Item 6 (Traceability) Non-compliant: the panel tells the operator on every run that a trade was logged to a CSV that no code anywhere writes, and nothing persists the decision object, so the walk back from output to raw data survives only as long as the process does. The DEFECT row from Entry #21 was resolved in favour of the stricter reading — the four-item gate stands, the three-item list is stale shorthand from before Revision 6 — which is the answer Claude expected, reached by an argument Claude had not made. Claude disagrees with the auditor on two points, both handed to Viktor: that Item 6 warrants Critical rather than Major, since a false claim reaches the operator every run; and that the reasoning behind Item 2's Compliant is stated too strongly, because backward-fill applied to input prices does reach the final bar through recursive indicators, even if the magnitude is negligible. One further correction belongs here, because it is Claude's and it came within an hour of the first. Claude initially reported that one of the auditor's citations was invented: Kimi had written that the log's Insufficient data entries show the weak length check firing on the 1h and 1w scans, and a search of `phase7_engine.log` returned nothing. The entries are in `scan_summary_report.txt`, the other evidence file supplied, where the string appears fourteen times, every one of them directly beneath an ASSET / TIMEFRAME header reading 1h or 1w. The claim was true in full, including the detail Claude called baseless; the auditor's only error was naming the wrong file, and its reasoning trace has it right. Claude had searched one file and concluded from that search that a string did not exist — the identical mistake to the Layer 5 error above, made a second time within the same day, and the second time used to accuse someone else of fabricating evidence. The accusation is withdrawn. Recorded rather than quietly amended, because a log that edits out its own false accusations is worth less than one that carries them.

Run B covered the seventeen Tier 1 invariants outside the gate. Kimi K3 returned eight Compliant, eight Non-compliant and one Unknown, with two rated Critical: Item 11 (No Circular Reasoning) and Item 13 (Fail Safely). Claude checked every substantive claim against the source and they hold. Item 11 is the strongest finding of the whole audit: `trend_health` is 30% of `bias_score`, then enters confidence a second time directly at 0.3 while `bias_strength` already carries it, then forms the entire base of `validation_score`, whose `validation_state` gates the decision — and the panel presents the result as four agreeing signals. Item 10 turned out worse than described: `panel_render.py` renders the same `trend_health` figure on the TREND line, again on the MOMENTUM line directly beneath it, and a third time as Current Market. Item 16 confirmed exactly: Bollinger Bands, KAMA, Typical_Price and the DIP indicator have zero consumers anywhere outside the file that computes them. Two small precision errors, both in the same class Claude has now made twice: Kimi wrote that it verified Item 17 by finding no backtest token in the bundle, and there are three, all in prose about the EV line rather than in any backtesting module — right conclusion, false stated method. The more consequential problem is what Run B did not find. Its Item 14 finding covers the `risk_score` aliasing and the `validation_state` gate, both real, but it never reaches `risk_model.py` line 75, where `bias_factor = 1.0 - abs(bias_score)/300` makes directional conviction tighten the stop distance — Item 14's prohibition written as arithmetic, reaching the stop price on the panel — nor the adjacent call in `engine_core.py` that omits `volatility_state` entirely, leaving the HIGH and EXTREME volatility tiers permanently inert. Both of those are in Kimi's own truncated first attempt, where it called the first one a textbook Item 14 violation. The run that ran out of tokens went deeper than the two that completed. That is the real cost of the truncation, and it is an argument for a second auditor over the same items rather than more items per auditor. Claude disputes two Compliant ratings, both for the same reason and both handed to Viktor: the indicator cache key in `engine_core.py` includes the last close price but not the bar's high, low or volume, so two fetches of an updating live candle can collide and serve stale indicators as current — undocumented decision-affecting state, reachable through the module-level singleton in `live_trading.py`, which is Item 4 and Item 12 rather than the Compliant both were given. Kimi has now examined that exact line three times across three gradings and judged it against a lens where it does not bite each time. DeepSeek, working blind with no register at all, found it on first reading.

“the freeze lifts once every Tier 1 item has a recorded finding — Compliant, Non-compliant, or Unknown — regardless of whether fixes have landed yet”

That definition exists because Reviewer 4 — Claude (Opus 5), in a separate session — objected during Revision 7 that “the audit has run” was an undefined trigger sitting on the rule that governs every other rule's stability. Next Steps was given the mechanical definition above in response. With Run A covering the four gate items and Run B the remaining seventeen, all twenty-one Tier 1 invariants now have recorded findings, and the condition is met. Ten Compliant, ten Non-compliant, one Unknown. It is worth naming what this does and does not do. It does not resolve anything: three Critical findings stand unfixed, four adjudications are open, and not one line of engine code has changed. It does not vindicate the register either — half of it came back Non-compliant, which is the register working rather than the engine passing. What it permits is proposing amendments through the process the Constitution already describes, which is a short note stating what changes, why, and what it deliberately leaves untouched, with Claude proposing and Viktor deciding, each one getting its own dated row. Three candidates are already waiting: the MVA gate wording the DEFECT row identified and the audit resolved, the blind-review method that produced Run 1 and is nowhere in the document, and Copilot's six candidate principles that have sat untouched in the appendix since Revision 1. None of them should be adopted today. The freeze was never about whether changes were good ideas — it was about not letting the document be revised by the same enthusiasm that wrote it, before anything had tested it. Something has now tested it, and the useful next act is fixing the ten Non-compliances rather than reopening the rulebook that found them.

Run C graded the twenty-three items in Tiers 2, 3 and 4, and with it every rule in the register has a recorded finding. Tier 2 came back four Non-compliant to three Compliant. The sharpest is T2-1: `calculate_dynamic_bias` in `bias_engine.py`, on finding a NaN, rewrites `close`, `EMA_20`, `EMA_50` and `RSI` directly on the `DataFrame` its caller passed in — no copy — and `engine_core` keeps using that frame for entry, risk and exit afterwards. Verified at lines 90 to 100. The path is unlikely to fire because `indicators` cleans NaNs upstream, but a module in the decision chain that can silently rewrite its own inputs defeats the boundary the whole audit relies on. T2-6 is the one to fix first because it is trivial: `requirements.txt` names five packages, two of which the code actually imports are missing (`requests`, `colorama`) and one of which nothing imports at all (`ccxt`, an order-execution library). A clean install cannot start the engine. Tier 3 returned three Compliant, three Non-compliant and four Unknown, and the Unknowns are the honest kind — process leaves no trace in a code snapshot, and the auditor said so rather than inferring discipline from tidy-looking labels. Tier 4 was graded as preferences and mostly honoured. The method differed from the earlier runs in a way that has to be recorded: Run C had web access and used it, fetching the public repository to check the one package claim that was externally verifiable. Runs 1, A and B did not. That makes T3-6's Compliant rest partly on evidence outside the bundle, which is a strength for that finding and an inconsistency across the audit. Claude then made the same fetch independently and closed two of the Unknowns: there are no tags and no releases, so T3-7 is Non-compliant rather than Unknown; and there are twenty-three commits on master, so the history is not the flattened snapshot Run C feared and the T3-9 depth question resolves in the engine's favour. One contradiction between runs, and Run B has it right: Run C's T4-4 says the duplicated `TREND` and `MOMENTUM` panel fields have since been fixed. The label was fixed; the number was not. `panel_render.py` prints `trend_health` on the `TREND` line, again on the `MOMENTUM` line immediately beneath it, and a third time as `Current Market`. Run C read the old scan report and inferred the fix; Run B read the source. Same model, same file, opposite conclusions — which is the clearest argument yet that a second auditor over the same items is worth more than a first auditor over more items.

"The goal is not to make this document sound better. The goal is to determine whether it can safely govern the engineering of the system."

Viktor commissioned this one alone, on a model with no prior involvement, while Claude was unavailable — and checked first that doing so was permitted. It is an external assessment like Entry #19's, not part of Step 3, which is finished. The instruction was strong: explicitly adversarial, told not to assume good faith, told not to reward the document for sounding sophisticated, and told to evaluate what the text permits rather than what it evidently intends. It came back forty-one pages, twenty findings at Critical or High, and a verdict of **REQUIRES REVISION**. Two of those findings are correct and are adopted today. The first is that the freeze-lifting definition attached no consequence to what the audit found — the freeze lifted with ten Tier 1 items Non-compliant and three findings Critical, and nothing anywhere said the engine's output should not be acted on meanwhile. Claude helped write that definition and did not notice the gap. The second is that the amendment process, now live, would let a Tier 1 invariant be weakened by a short note reviewed as Claude proposes and Viktor decides — with the sharpest version being amendment by clarification, which is the exact argument shape Entry #21 already identified as the one the freeze existed to refuse. That finding is a criticism of an arrangement giving Claude considerable latitude over a document Claude largely wrote, and Claude thinks it is right anyway. Both are now in the Constitution, along with a front-matter sentence stating what the engine is. A third gap — Item 20 not naming crash-reporter capture of a process environment — is recorded and deliberately left unfixed, because closing it means amending a Tier 1 invariant and the rule just adopted requires that to be reviewed by someone who is not Claude. The new rule bit within the hour, on Claude. The rest is declined, and the reason is the reason Viktor named before either of them had read a finding: the review was instructed to treat this as a production-grade quantitative trading system, and it did, despite reading Item 1 and acknowledging in its own executive summary that the engine cannot execute. Roughly nineteen proposed new invariants follow from that framing — multiple-testing correction, calibration monitoring, an execution model covering fees, spread, slippage, latency and fills, market-structure handling for halts and delistings — every one of them correct for a system with a backtester and a live order path, and every one aimed at capabilities this engine does not have and is forbidden by Item 18 to acquire. Adopting them would take the register past sixty rules to govern a product that does not exist. They go to Future Amendment Candidates, to be revisited when the backtester is actually being built. Two structural lessons worth keeping. The instruction asked what invariants were missing and never asked which were unnecessary, so a hostile review under it can only ratchet upward; a future version needs a subtraction question. And the framing in an instruction outranks the content of the artefact — the reviewer had the correct definition of the engine in front of it, said so, and audited against the instruction regardless. The document now states what it is on page one rather than page fifteen.

"If it fails on your machine, the test is wrong before the engine is."

There were no tests. Not thin coverage — zero: `test_live.py` contained no assertions and `requirements.txt` named no test framework, while the runtime log recorded seven distinct classes of failure across nine occasions where a change was accepted and then found broken by running the engine by hand. The Constitution's Step 7 says "regression test after each fix," which was not a step that could be performed. Eighteen tests now exist: a compile-and-import check across all twenty files, a clean-checkout test, eight corrupted-data fixtures for Item 3, a golden-path snapshot against a pinned deterministic dataset, a determinism check, and two symbol-hardcoding tests. Five pass and thirteen fail, and the failures are the point — they are the Non-compliances written as executable acceptance criteria, each going green when its fix lands. All seven historical crash classes were reintroduced one at a time and the suite caught every one. Two things are worth recording beyond the artefact. The first is that `pytest` could not be installed in the authoring environment, so the suite was written as plain assert functions with a dependency-free runner alongside — which turned out to be the right shape anyway, since the suite has to work on a clean machine before `pip install` has succeeded, which is the exact situation the dependency test is about. The second is that the golden-path test was shipped unverified with the sentence above written into it, and then failed. The test was wrong, not the engine: it called `Phase7Engine.run()` directly, bypassing `SignalRouter` and therefore the entire decision layer, and returned the same dataframe for every symbol so the asset correlated with itself at +1.00. Both fixed, and the revision history is written into the file's own docstring rather than tidied away.

"A different method beat a different model."

The harness was built to make fixes safe. Its first real run found defects instead. **One severity escalation:** the published repository does not start from a fresh clone. `main.py` builds its log handler at module scope on line 16 and creates the `Logs/` directory inside `main()` on line 41; `FileHandler` opens its file eagerly, so a clone without that directory raises `FileNotFoundError` during import — before `main()` runs, so the `try/except` inside it cannot catch it. Run 1 found the ordering and described it as soft: "the logging machinery catches it and prints to `stderr`." That is not what happens. The finding was real and its severity was understated by a full category, invisible on the machine the engine was built on because `Logs/` has existed there since the first run. **Two findings nobody had:** the string `AERO` is hardcoded into user-facing reasoning text at `decision_model.py` lines 411, 413 and 419 and `panel_render.py`: 83, so running the engine on `SOLUSDT` produces an explanation that talks about `AERO`, and running it on `BTCUSDT` produces one claiming to compare `AERO` against `BTC` while comparing `BTC` to itself — which contradicts Run C's Compliant rating on T4-2, true of the arithmetic and false of the prose; and `decision_model.py`: 419 appends "relationship" to a label already ending in it, printing "a weak / no clear relationship relationship" to the trader. Four audit passes across three models read this code carefully and found none of the three. The reason is not that they read it badly. It is that reading is a different instrument from running, and the project had been buying more of the first while owning none of the second. That is the finding worth keeping from Phase A — more than the eighteen tests.

“Different lineage is necessary for judgment independence; it was never sufficient.”

Recorded because the audit's whole architecture rests on it and it had never been written down in one place. Nine models have now seen the Constitution: Claude as its author, then Copilot, Gemini, ChatGPT, Claude Opus 5 in the Reviewer 4 role, Grok, Kimi K3, GPT-5.6 Luna Pro for the hostile review, and GLM 5.3 for Step 5. Four have seen the engine source in a graded capacity — Claude, Kimi K3, DeepSeek V4 Pro and GLM 5.3 — and three more saw it during the build itself: Claude Sonnet 4, DeepSeek V3 and DeepSeek R1, through Aider. That last group is the one that matters, because it means DeepSeek's lineage had prior exposure to this codebase, and Run 1's claim of no prior involvement is therefore weaker than it was stated to be. Independence is tracked at the lab, not the checkpoint: a newer model from a family that has already worked on the artefact is not an independent reviewer of it, and treating a version bump as a reset would make the safeguard ceremonial. Still clean on both lists: Meta, Mistral, Qwen, Cohere, Amazon, MiniMax. One consequence already acted on — Luna Pro was considered for Step 5 and rejected, because it had seen the Constitution during the hostile review, and GLM was chosen on the basis of being clean on both lists at the time. It no longer is. Every run spends independence that cannot be recovered, which is an argument for deciding what a reviewer is for before opening the room.

"ML feels like the milestone that makes the project serious. That instinct is what this document exists to catch."

Viktor asked what could be counterproductive about adding machine learning — the obvious use being to fit the six bias weights against real outcomes rather than choosing them by hand — and then parked it. Not cancelled; parked, with conditions, which is the difference between a decision and a mood. The argument, verified against source before it was given: **training weaponises a dormant bug**. Item 2 is Compliant, but input-side `bfill` reaches the final bar through recursive indicators. Live mode only ever evaluates the current bar, so the leak has nothing future to pull from and stays inert. Training evaluates thousands of historical decision points, and at each one the backward fill pulls from bars that had not yet happened. The result is an excellent fit with no live edge, and the failure is silent — a good-looking equity curve, which is the most expensive kind of wrong this project could manufacture. **The training data does not exist**: `live_trading.py:210` does persist a decision, but nothing anywhere records what price did afterward and `engine_core.py:466` overwrites its state file each run rather than accumulating. Decisions without outcomes are unlabelled examples. **Item 11 makes fitted weights worse, not better**: fitting over features that are secretly the same feature produces large offsetting coefficients that swing on tiny data changes — crude weights wearing a lab coat. **Item 13's sentinels become learned signal**: `RSI=50` means the indicator broke, and a model would learn to predict from it. Two costs that are not bugs: reproducibility gains a whole new surface, since a decision would then depend on a weights file depending on a training run depending on a data snapshot, all needing version-pinning to the decisions they influenced; and explainability drops, which matters for a tool whose entire job is helping a human judge. Revisiting requires all five of: Items 2, 3, 11 and 13 fixed and verified by the harness; a working decision *and* outcome log with accumulated history; a backtester validated against known-answer cases; a kill condition written before the first fit; and an out-of-sample test that beats the current hand weights. If the fit does not beat 0.30 / 0.20 / 0.15 / 0.15 / 0.10 / 0.10 on data it never saw, that is a real result — it means the hand weights were fine.

#33 Step 5 — The Remediation Sequence, and What Verifying It Found

August 29, 2026

AUDIT STEP 5 — VERIFIED

“A fix made before its apparatus exists can be made, never resolved.”

Step 5 ran on GLM 5.3 — the pinned slug, not the floating `glm-latest` alias, which silently repoints when a successor ships and would have quietly poisoned this document's record of which model produced what. The instruction was Viktor's, not Claude's: it asked for position, dependencies, effort, risk-of-doing and risk-of-not per item, five direct questions, and an explicit flag wherever the reviewer's reasoning might be correlated with the model family that built the engine. Two edits before it went out — the build-timeline phrase removed, and a paragraph added about Phase A, because the instruction predated the harness and would otherwise have hidden three real findings from the reviewer. What came back organises sixteen items by verifiability prerequisite and shared edit seam rather than by severity, on the reasoning that severity decides membership in the gate sets and not position. Claude spot-checked every claim that could be checked against source; nine of nine held, including that `requests` and `colorama` are imported and undeclared while `ccxt` is declared and appears nowhere else in the codebase, that `close_time` arrives in the raw response and is discarded one line later so any staleness check must precede that line, that `engine_version` exists in config and is written nowhere, and that the indicator cache can never hit across runs because its key embeds the last close and the dict is per-process. Two things came out of the verification rather than the plan. The codebase contains two comments acknowledging previous instances of the same defect class — a gate comparing against a string literal no producer ever emits — which makes the dead `trend_failure` gate the third instance and argues for a class-level check rather than a third individual fix. And Claude came within one sentence of telling Viktor the plan understated that fix, on the grounds that the `STRUCTURE` column is never assigned — it is, at `structure.py` line 4502 of the audit bundle, via a `.loc` form the first search pattern did not match. That would have been the third false assertion of absence on this project. The exhaustive re-search caught it; the pattern is now a working note.

#34 The Machine Was Rebuilt, and the Baseline Survived It

August 30, 2026

REFERENCE — FILED FOR THE RECORD

“A golden baseline is a claim about an environment, and the environment was destroyed.”

Windows was reinstalled and every drive wiped. Nothing of value was lost: the repository stood at 375334a on GitHub and nothing lived only on disk except the gitignored `logs/` tree. The reason this earns an entry rather than a shrug is Item 5. The golden snapshot is not a claim about the code alone; it is a claim about the code *in an environment*, and that environment was destroyed and rebuilt from nothing.

`docs/audit_package/environment_before_reinstall.txt` records the library versions and the Python 3.12.0 that produced the original baseline. The rebuild resolved different ones — `numpy` moved from 1.26.4 to 2.2.6, `pandas` and `pandas-ta` unchanged — and the snapshot did not move. That was checked rather than assumed, and it is a real result. It is also the strongest available argument for what became audit Finding 15: **nothing in the repository required those versions**. The baseline held across a major version bump of the numerical library underneath every indicator, and it held by luck that nobody had arranged. Viktor's machine is now on Python 3.12.10 and the remediation sandbox runs 3.12.3; both produce identical suite results, verified on every delivery since.

"It tests that selected implementation details have not changed more strongly than it tests whether the engine is correct."

Sequence item 16 executed. GPT-5.6 Luna Pro received the engine source, the test suite, the frozen audit copy of the register and a written instruction (docs/audit_package/item16_review_instruction.md), and returned docs/audit_package/luna_pro_audit_report.md — a full 44-rule verdict table, fifteen findings, a test-suite assessment and a release-gate determination. **Release gate: not met.** Five findings rated Critical: Item 3, abnormal volume reaching analysis unchecked; Items 8 and 13 together, a failed macro input rendered as an ordinary neutral reading; Item 13, partially invalid indicator columns passing without degradation; Item 11, bias and confidence reusing derived evidence as if it were independent; and Item 14, AGGRESSIVE selected from conviction and entry quality with no independent risk decision. Ten further findings at Major or Moderate, and twelve rules graded Not verifiable. Two things about this run belong in the record rather than in a summary of it. First, the sentence quoted above — the auditor's judgement of the suite — is the sharpest single observation any reviewer has produced about this project, and it was aimed at a harness built four days earlier specifically to be better than that. Second, see Entry #41: the model that produced this report was not on the clean list the Remediation Plan had drawn up for this step.

"Under pytest, a return is a pass."

Two batches, one commit — dba1b63. **Batch 1, the skip mechanism.** The suite contained 46 occurrences of `if not _engine_available(): return`. A returning test is a passing test, so on a machine without `pandas_ta` the suite reported success for work it had not done — audit Finding 13, and precisely the defect Entry #29's harness had been built to prevent. All 46 became real `pytest.skip()` calls. Running the suite without `pandas_ta` then reported 46 SKIPPED instead of 46 false passes, which is how four tests that import `core.engine_core` directly, bypassing the guard, were found to fail outright in that environment. Recorded at the time and deliberately not fixed in the same batch — see Entry #38. **Batch 2, five unambiguous fixes.** The "full size" text in `decision_model.py` (Finding 8, path B); `main.py`'s bare 'Logs' (Part 6 observation 2); `trade_quality_current` and the vacuous assertion that guarded it (Findings 10 and 13); the hardcoded `Lookback 8` on the panel, now interpolated from `config.STRUCT_LOOKBACK` (Finding 8, path A); and `np.isfinite` in risk validation. **None of those five named the audit finding it closed.** Three whole findings were later discovered to be already shut for exactly that reason — Entry #40 records the cost from the other side.

“Decide items 3, 11, 14 myself.”

Viktor's instruction, delegating all three trading-judgment calls rather than ruling on each in turn. Recorded as a delegation because Roles & Authority assigns this class of decision to Viktor, and this is the second time he has knowingly handed a specific class of them over. Commit c4dfcc7. **Item 3 — abnormal volume: reject, degrade or accept, per case.** All-zero volume is now rejected at `data/validation.py`, because there is no measurement from which to build a volume-weighted read. An isolated extreme spike is still accepted at that layer: the original “deliberately not implemented” reasoning held, in that a spike is real data and rejecting a run over a busy market makes the engine least available exactly when it matters most. What changed is that a spike no longer reaches every downstream score unflagged — `indicators.py` detects one above ten times the recent rolling median and records it as a degradation, capping confidence rather than substituting a value. **Item 11 — remove the duplicated factors rather than argue for their independence.** The earlier sequence-item-11 pass had removed one duplicated term and left three siblings standing. `bias_score` is now the one place all six weighted factors are combined and confidence is exactly its magnitude; `continuation_strength` no longer carries a trend-health-derived component; and `bias_engine.py` gained the explicit dependency-graph comment the audit's required action had actually asked for. **Item 14 — the labels survive, gated.** `classify_risk_regime()` already computed a four-tier regime, and only the EXTREME-or-not boolean ever escaped `validate_risk_parameters`. The regime is now threaded through as its own contract field, and AGGRESSIVE is refused when it is HIGH VOLATILITY RISK or worse. Direction and whether a trade is permitted at all are untouched; this gates intensity only. Ten net new tests. The golden snapshot moved, in exactly the four fields predicted before the run.

“A test docstring naming what it did not fix is a debt with an address on it.”

Four commits. **Item 8/13, the macro half (5d2cbb6)**. A failed macro-timeframe fetch left `macro_bias` at its initialised "NEUTRAL" with nothing added to degradation, so a failed higher-timeframe read and a genuinely flat one rendered identically on the panel. Both the validation-failure path and the processing-exception path now record a degradation. `test_the_macro_series_is_actually_read` had documented this in its own docstring as “recorded rather than fixed... a rider on sequence item 9's degrade ruling”; that assertion was rewritten alongside the fix, which is what the docstring had asked whoever fixed it to do. **Finding 15, dependency pinning (a26c545)**. `requirements.txt` and `requirements-dev.txt` now pin exact versions, verified by building a virtualenv from empty and confirming `pip freeze` matched before the suite was run. Closes the risk Entry #34 records the project having survived by luck. **The four unguarded tests (3d0b410)**. The ones Entry #36 recorded and left. Three are entirely about `Phase7Engine` and now carry the same `_engine_available()` guard as the rest of the suite. The fourth, `test_every_module_imports`, needed a different fix: it checks all 21 engine modules for import-time defects and only three of them need `pandas_ta`, so a blanket skip would have stopped checking the other eighteen for an unrelated reason. It now excuses that one exception by name and still fails on anything else. **Stale labels (2be405f)**. Two golden-path tests still carried “EXPECTED TO FAIL until sequence item 12” in their docstrings; item 12 had fixed both defects before the Luna Pro audit began and both tests had been passing ever since. Documentation only — but a docstring that lies about the state of the code is the same class of defect as a panel line that does.

“A derived document cannot tell you what it never contained.”

Found by reading `luna_pro_audit_report.md` end to end rather than the roadmap written from it. Finding 3 — Item 13, partially invalid indicator columns passing without degradation — is one of the audit's five Criticals and had never been entered into any roadmap, so it was never scheduled, never ruled on, and never noticed missing. From 31 August to 2 September `PHASE7_NEXT.md` stated that remediation of the five Criticals was complete. Four of them were. Commit 108cc9f. **The defect.** Every indicator guard asked one question, `.isna().all()` — did the calculation return nothing at all. That cannot catch a series with 299 good values and no value at the bar the decision is made on, and it could not even in principle: `clean_series(method="forward_fill")` had already filled that gap with the previous bar's number before the guard ran. **The class was wider than the two instances the audit named.** Injecting a trailing NaN into each indicator in turn, before the fix: ATR, RSI, ADX, SuperTrend and both EMAs — every one carried a stale prior-bar value into the decision row, and not one recorded a failure. Two had no guard at all to fix, the SuperTrend *level* and the EMAs. The same trailing fill ran on the raw OHLCV columns, turning a truncated final candle into a synthetic bar repeating the previous close. **And the consumers were re-fabricating what sequence item 9a had removed.** A missing RSI fell back to `50.0`, inside the “not extended” band, scoring the full 15 of 15 — while `indicators.py`'s own failure text told the operator it scored 0 of 15. A missing HVN fell back to `close`, making the distance exactly zero and scoring 12 of 12; that is byte-for-byte the defect item 3 had fixed for VWMA six days earlier, sitting forty lines above it. A missing ATR fell back to `close * 0.02`, the flat constant item 9a had deleted — on the pinned fixture, 0.016035 against a real 0.010554, a 52% overstatement of the distance that sets the stop. **Ruled:** a value not measured at the decision bar is absent, and absent means that indicator failed — which hands it to the degradation machinery that already exists. No new policy was invented; the existing one was applied one row over. Verified end to end: before the fix, a trailing-NaN ATR produced `degraded: False, missing_inputs: [],` a full stop and three targets, and moved entry quality from 45.18 to 45.25 — a different answer, reported clean. Eighteen new tests, fourteen of which fail against the pre-fix code; the other four are controls that must pass on both sides. The golden snapshot did not move, predicted in advance and for the right reason: real pinned data has zero trailing NaNs, which is why this went unseen. **Recorded against Claude:** the first draft of that test file reintroduced the defect fixed in 3d0b410 the same morning — five of its tests imported `indicators.indicators` with no `_engine_available()` guard and errored instead of skipping. Caught by running the suite in a `pandas_ta`-free virtualenv before packaging. Same class, same day, inside the fix for the class.

#40

The Status Sweep, and a Count That Was Wrong by Three

September 2, 2026

OBSERVATION — FILED FOR
THE RECORD

“Neither document knows the state of the code.”

Claude told Viktor on 1 September that nine Major and Moderate findings remained open. That number came from the audit report rather than from the code, and it was wrong by three. Checked by opening each location the audit quoted rather than trusting the summary: **Finding 8** (inaccurate user-facing claims) was fully closed — Lookback interpolated from config and the “full size” text gone, both in batch 2, with the macro path closed by 5d2cbbbe. **Finding 10** (the unconsumed trade_quality_current) was closed in batch 2. **Finding 15** was closed at a26c545 the previous day. None of those three commits mentioned a finding number, which is why nobody knew they had shut one; Entry #36 records the same omission from the authoring side. Genuinely open: Findings 6, 7, 9, 11 and 12 in full, plus the remainders of 13 (the plotting test still asserts only the absence of ERROR records) and 14 (the required-shape assertion still omits provenance, degradation and the log path). Seven, not nine. The status table now in PHASE7_NEXT.md cites a file and a line for every verdict, so the next reader can re-check it rather than trust it. This is the mirror image of Entry #39: that one was the plan missing what the report had, and this one is the report not knowing what had been fixed since.

#41

The Step 8 Auditor Was Not on the Clean List

September 2, 2026

OBSERVATION — FILED FOR
THE RECORD

“The party that made the error should not be the party that decides the evidence of it disappears.”

Phase7_Remediation_Plan.pdf, 29 August, records that Luna Pro was considered and rejected for Step 5 because it had already seen the Constitution during the hostile review, and names the six families still clean on both lists for the Step 8 re-audit: Meta, Mistral, Qwen, Cohere, Amazon and MiniMax. **Step 8 was run on Luna Pro the following day.** Entry #31 sets out why the ledger exists — independence is tracked at the lab, not the checkpoint — and Entries #18 and #20 record Grok being removed from the auditor plan for precisely this reason, having read the Constitution in an earlier conversation. The same disqualifier applied here and the ledger was not consulted. **What this does not mean.** The findings are sound. Every one was checked against source before any of it was fixed, and Finding 3 was verified by running the engine rather than by trusting the report. Nothing here argues for discarding the report or redoing the work it produced. **What it costs** is the one thing the ledger buys: the next re-audit cannot treat agreement with this one as independent corroboration, because it would be the second reading by a model that had already read the standard. The next re-audit should go to one of the six families named above, and its agreement with Luna Pro should be read as agreement between two readings, one of them contaminated — not as confirmation. Recorded rather than quietly noted, on Entry #21's reasoning.

“A document you have sampled is a document you have not read.”

Viktor asked whether the Constitution needed upgrading. Answering that honestly required reading all nine governing documents rather than sampling them — Entry #17's lesson, and rule 17 of PHASE7_NEXT.md's earned list, which exists because Claude once declared this exact PDF clean after searching four guessed phrases. **The answer is no, and the project had already ruled so.** Entry #26, written the hour the freeze lifted: the useful next act is fixing the Non-compliances rather than reopening the rulebook that found them. Four things the read surfaced that were in no working document. **One: the release gate is in force and blocks on Item 6.** Viktor's 29 August ruling raised Traceability from Major to Critical; Item 6 is the audit's Finding 7, still open. Until it lands *and is re-audited*, no output may be relied on for a real trading decision, and the backtest architecture is not built. Nothing in PHASE7_NEXT.md had said so. **Two: the adjudications were ruled on 29 August**, in Roadmap Revision 4, whose title says exactly that. Claude reported four still open, having read the Engineering Notes and the Constitution — both of which stop before that ruling — and not the Roadmap. Only the Item 20 amendment remains. **Three: Item 20's gap is already served operationally.**

Phase7_Credential_Security_Protocol.pdf §6.2 covers telemetry, crash reporting, error aggregation and usage analytics, and is tagged “Enforces: Tier 1, Items 20 and 21” — written the same day as the invariant whose channel list omits it. The pending amendment tidies the register to match a practice that already exists; it does not close a live hole, and the engine holds no credentials at all. Unblocking it needs one uninvolved model given a text extract, not the PDF that Gemini's and Copilot's classifiers refused. **Four: the documentation is a week in arrears** — this log stopping at #33 being the largest part of it, alongside no Change Impact Records since the standard took effect on 25 August, no Version History row for Step 8, and two companion documents still pointing at Rev 6. **One check nobody has run.** The item-16 instruction requires the re-audit to grade against the frozen audit copy rather than the live version annotated with the outcomes of a previous audit.

Phase7_Constitution_v1.0_RATIFIED_AUDITCOPY.pdf is 69,656 bytes against the live document's 112,290; the smaller size is consistent with a frozen copy and there is no reason for alarm. It is still worth opening once to confirm it carries no AUDITED, AMENDED or DEFECT row, because it is a one-minute check on whether the re-audit graded the exam or the answer key. *(An earlier draft of this entry compared that byte count against the 101,831 recorded in Phase7_Audit_Execution_Instructions.pdf and called it a mismatch. That was wrong — those instructions describe the Step 3 package for Kimi K3, a different audit with a different material set. The claim was made to Viktor before it was checked. Corrected here rather than deleted.)*

“A hash proves the input changed. Only an archive lets you rebuild it.”

Item 6, Traceability, which Viktor's 29 August ruling raised from Major to Critical — and therefore the finding holding both the release gate and the backtest gate shut. Commit 302db8b. **The gap.** Sequence item 12 built a decision log: what the engine concluded, plus a five-field fingerprint of what it saw. The fingerprint was a last-candle timestamp and a row count, and two different frames can share both. Nothing stored told them apart, so “reconstructable” was a word in the Constitution rather than a property of the engine. **Viktor's ruling: hash AND archive, pruned at ninety days.** The Constitution says under Item 5 that the retention decision is one “the audit should force explicitly rather than leave implicit”; this is that decision made rather than assumed. The two halves do different jobs and the difference is the whole design. **The hash detects** — it costs nothing, it lives in the log, and the log is never pruned, so a decision from two years ago can still be checked against data fetched today. **The archive reconstructs** — the actual candles, the only thing that can rebuild a run whose source has since changed, and the only part with a cost, which is why it is the only part with a limit. About 31 KB a run; roughly 2.8 MB steady-state at the cap. Past the window a run does not become unverifiable — it becomes *verifiable but not rebuildable*, and the record says which by whether the file is still there. **Two design points worth keeping.** The hash is taken on the OHLCV as validated and *before* any indicator column exists: hashing the final frame would fingerprint this engine's own arithmetic together with the market data, so changing an indicator length would present as the exchange having changed its candles. And the canonical form is written out explicitly rather than leaning on `to_csv`, because pandas has changed its float repr and NaN spelling across versions this project has already run on — Entry #34 records numpy moving 1.26 to 2.2 across the machine rebuild alone. **Also closed:** the six bias weights live in `bias_engine.py`, not `config`, so changing 0.30 to 0.35 changed every decision the engine makes while leaving two runs byte-identical in the log. They are now fingerprinted, read live from the module rather than restated. **Verification.** Fifteen tests, and the central one does not inspect fields — a test asserting “lineage” in `decision` would pass just as happily over a section full of nulls, which is precisely the shape Luna Pro's assessment of this suite warned about. Instead it takes the archive, rebuilds the candles from it, runs the engine again against the rebuilt data and nothing else, and requires the identical decision. The golden snapshot moved exactly as predicted before the run — lineage added, provenance gained seven keys, and not one existing value changed. This commit adds a record and alters no output.

“Every hash matched. Every number matched. The whole difference was one separator.”

The Findings 6 and 7 work built the archive path with `os.path.join`, which returns a forward slash on Linux and a backslash on Windows. That path is written into the decision log, so the same run archived on the two platforms recorded two different strings for one file — and the golden snapshot, baselined in a Linux sandbox, could only ever match on Linux. It failed on the first run on Viktor's machine. **Two things about it are worth more than the fix.** The first is the near miss: `decision_log.log_path()` has the identical shape and escapes the bug by luck rather than design — `config.LOG_DIR` already ends in a separator, so its single join never inserts one. The archive path joins a directory level deeper, which is where the backslash appeared. Existing code working was not evidence that the pattern was safe, and it was read as if it were. The second is why nothing caught it earlier: **the entire verification for that commit ran on Linux, where the bug is invisible**, and a golden snapshot only ever compares a value against itself on whatever platform baselined it. The patch had been called “verified against a fresh checkout” — true, and on one platform. Fixed by normalising to forward slashes for both the file I/O and the recorded value, since Python accepts them on Windows and one spelling then serves both purposes. Pinned by a test that fails on Linux too if it regresses. Recorded as its own entry rather than folded into #43, because a defect that reached the operator's machine inside a commit whose message claimed thorough verification is not a footnote to that commit.

“Reading finds what is written. Running finds what happens.”

Viktor ran `python main.py` against live MEXC data the evening Findings 6 and 7 landed. AEROUSDT 4h came back bearish on every measure the engine has — bearish bias, bearish regime, bearish structure, an LH-LL sequence, strong bearish distribution, SuperTrend freshly flipped — with one dissenting reading, `MACRO TREND: BULLISH`. It printed **DECISION: CONSERVATIVE LONG**, with the stop at \$0.4889 above a price of \$0.4725 and three targets descending to \$0.4233. **A long label on a short plan.** Every number in that plan was correctly computed from a correct bearish analysis; the word attached to them was not. An operator following the `DECISION` line would have bought an instrument the engine had just analysed, in detail and correctly, as a short. It also printed “Bias is bullish and the broader macro trend agrees” four lines above its own Validation Note reading “The higher timeframe disagrees with this bias.” **Two causes.** `decision_model.py` opened a direction from any of three independent sources — `raw_bias` or `long_signal` or `macro_bias` — so the macro clause alone was enough; and `trend_health >= 50` then passed because trend health is an `UNSIGNED` magnitude, so a strong bearish trend scores 69 and no bearish evidence anywhere in the run could block it. The bearish block below never ran, because the bullish one returned first. Meanwhile `risk_model.py` builds stop and targets from `detailed_bias` alone. Two direction sources, never reconciled, in two modules neither of which knew the other existed. **Viktor ruled: bias is the sole direction source.** Macro keeps its existing 10% vote inside `bias_score` and gets no second, overriding one — letting it override the blend counts one piece of evidence twice, which is Item 11 in the module that picks the side. Entry-zone signals lose the same privilege for the same reason. Two further instances surfaced in testing that nobody had seen: the mirror case returned `CONSERVATIVE SHORT`, and a `NEUTRAL` bias with a long entry signal returned `AGGRESSIVE LONG` — the most confident label the engine has, from no directional view at all. **And a guard, because narrowing a source is not the same as checking.** `_refuse_incoherent_plan()` reads direction off the targets themselves and refuses any action whose label contradicts its own levels. Deliberately a refusal rather than a relabelling: one of the two sources is wrong and nothing inside that function can tell which, so `NO-TRADE` is the only answer available that is certainly not the wrong one. **What this says about the audit.** Luna Pro read the source and the tests and did not find this. Neither did four earlier passes across three models. It needs bias and macro to actually disagree on live data, and no pinned fixture makes them.

“A disk problem must neither destroy an analysis nor veto one.”

Found while writing the halt-safety test for the raw-input archive, and deliberately left unfixed in that commit: fixing it there would have let that test pass for a reason unrelated to what it was written to prove. **The cause was smaller than “the engine cannot log.”** `route()` opened with two unguarded `os.makedirs` calls that wrote nothing. Every writer in this engine already creates its own directory on demand inside its own error handling — `decision_log.write` returns `None`, `_save_state` warns, `plot_engine_chart` warns, `lineage.write_archive` returns `None`. The two calls duplicated all four and added a failure mode at the worst point in the run: the top of `route()`, before anything had been computed. An unwritable log directory raised there, the broad handler reported “Router execution failed”, and the operator lost the entire analysis rather than merely the log of it. Four independently recoverable conditions collapsed into one total failure. Same class as sequence item 14's own `REQUIRED_DIRS` finding, which removed the list and left these two calls standing — earned rule 22 collecting on the item that wrote it. **Viktor ruled: a run whose decision log cannot be written still authorizes a trade.** It warns, the panel makes no claim that anything was logged, and the operator decides. His 29 August degrade-not-halt ruling applied literally. **Claude recommended the opposite and was overruled.** The argument made and rejected: Item 6 is Critical, and a trade taken on a decision that left no trace is unauditably by construction — which differs from a failed *archive*, where the hash still lands in the log and the run stays verifiable even though it is no longer rebuildable. When the log itself fails there is no record at all. Recorded here because the difference matters to whoever audits this next: a ruling with its trade-off on the table reads differently from an absence, and the cost is stated plainly — the one decision an operator acts on without a record is the one an auditor would ask about first. The tests pin the ruling and say so in their own docstrings, so a later reader who thinks the assertion looks wrong finds the reasoning instead of guessing. **Categorisation note:** a failed write is not added to degradation. That list blocks trading and is about inputs the ANALYSIS was computed without. Nothing here was missing from the analysis — only from the filing of it.

"I really should have run the engine more often, but I was too caught up in the workflow."

Viktor's observation after Entry #45. The better answer than resolving to be more diligent — which decays — is to make the suite able to reach the state, so that it does not depend on anyone remembering. **The reason nothing caught it is exact: no pinned fixture makes bias and macro disagree.** The committed series trends one way and its daily aggregate trends the same way, because the aggregate is built from it. A suite cannot reach a state its fixtures never enter, so a hundred and seventy tests passed over a defect that only exists in that state.

`tests/test_timeframe_disagreement.py` generates a series where the timeframes genuinely disagree: a long rally, then a sharp multi-day break, so the daily EMA-50 lags and still sits below the daily close while the 4h structure has decisively turned. An ordinary market condition of the kind that happens most weeks, not a contrivance built to trip a branch. Generated rather than committed, following `test_golden_path._write_pinned_set`'s discipline: one pure function, all three files derived from it, and no RNG anywhere — the wobble is `sin()`, so the series is byte-identical on every machine and every numpy, which matters on a project whose baseline has already survived a numpy major version by luck. **The parameters were chosen with margin rather than at the edge.** A 0.16 drop put the daily close within 0.0007 of its EMA-50; 0.14 leaves real daylight. A fixture that only just produces its condition stops producing it the first time an indicator length changes, and stops silently. **Two of the seven tests assert that the fixture still splits the timeframes,** before anything is asserted about behaviour — if it ever stops, every other test in the file would go on passing while testing agreement. That is section 7.3's "setup contradicts what it claims to test", and a file like this is exactly the shape that fails that way. The property pinned is deliberately not "the engine returns WAIT here" — that is today's answer to today's thresholds. What must never be true at any threshold is a LONG label above descending targets. Four of the seven fail against the pre-fix code, reproducing the live run from generated data.

"Nobody could have caught it by thinking harder. Only by reading the bill."

Viktor delegated this one — "This is your call and I trust you make the right decision" — so it is recorded with its reasoning rather than only its conclusion, and can be overturned on the argument. **The project held two positions and had not noticed they cannot both be true.** Entry #18 resolved Grok's prior exposure by running Step 3 "in a fresh conversation with no shared memory of the earlier one." The Remediation Plan, eleven days later, rejected Luna Pro for Step 5 *because* it had read the Constitution during the hostile review — the same kind of exposure, treated as permanent. Entry #41 then graded Step 8 against the stricter reading without noticing the looser one existed. Which answer you got depended on which document you opened. **Three different things were being recorded as one.** SESSION exposure is a model reading the document in a chat; TRAINING exposure is the artifact being in the weights; LINEAGE exposure is a sibling from the same lab having worked on it. Entry #31's rule — independence is tracked at the lab, not the checkpoint — was written for the second and third. Applying it to the first is a category error, and it is why the clean list emptied faster than it needed to. **Ruled: session exposure is cleared by a fresh conversation where training-on-conversations is off; training and lineage exposure never clear.** Where it cannot be established whether a session fed training, treat it as permanent. This ruling is LESS strict than the reading the project had been using, which is worth stating plainly: over-strictness had a measurable price here, in models spent for a reason a fresh conversation removes. Kimi K3 becomes available again. **Then the assumption was checked rather than assumed.** Viktor exported the OpenRouter activity log — 713 requests, every model, every route. All `variant=standard`; not one free endpoint in the entire history, so no material was ever sent anywhere that trains. It also dated the Step 3 truncation exactly: `completion=16,384`, `finish=length`, at 2^{14} output tokens — Entry #23's "default token ceiling" with a number on it. **And it showed the ledger was wrong.** Five models ran through Aider on 22–24 August while the engine was being built. Entry #31 names three of them and misses `mistral-nemo` (71 requests) and `claude-3-haiku`. Mistral's lineage worked on this codebase through exactly the mechanism used to disqualify DeepSeek, and had sat on the clean list of eligible auditors for a week. Entry #31's table was written from recollection. The correction matters less than how it was found: every provider bills per request, and the bill cannot be mistaken about what was called.

“Eight tools were enabled. One `git clone` would have handed over the answer key.”

The package the round-2 auditor receives is now **generated** by docs/build/build_audit_package.py rather than assembled by hand. Entry #24 records what hand assembly cost: “a false claim found in Claude's own Step 2a package.” A script that walks the repository and computes the manifest from the bytes it wrote cannot make that class of error, and it *refuses* to build if docs/ would ship, because intending to withhold the answers is not the same as withholding them. Three things Luna Pro never got: version-control history (metadata only — subject lines withheld, since “Audit Findings 6 and 7” leaks a finding number and its outcome in eleven words), the project files that decide how dependencies are pinned, and execution transcripts. Five rules came back Not verifiable in Step 8 *only* because history was withheld; that was a defect in the package, not the project. **The instruction's Rev 2 states two things the project would rather not say:** that Step 8's own auditor was not on the clean list, and that this repository is public, so any model trained since may hold the codebase in its weights — a channel the ledger cannot see and which applies to the chosen auditor as much as anyone. It also says plainly, in its own section, what this round cannot be: the fixes shipped with comments and docstrings describing the defects in detail, so this is not a blind re-discovery but a check of whether the claimed fixes are real. **And the near miss.** The run started with eight tools enabled, including a shell, web search and web fetch. Nobody chose that and nobody checked. It probed an empty sandbox and found nothing — but the repository is public, and one command would have handed over PHASE7_NEXT.md, the previous audit report, the Engineering Notes and every commit message. Three further tools were worse in a quieter way: Fusion, Advisor and Subagent would each have silently put other models inside the audit, leaving the report labelled with one model's name while being partly the work of models nobody chose. It was caught by looking at a screenshot at the right moment. That is luck, and earned rule 28 — written four hours earlier the same day — says luck is the signal to change the setup rather than to be more careful next time. A pre-flight checklist now exists, and the omission was Claude's: the setup walkthrough covered model, provider, max tokens, streaming, file parser and data retention, and never mentioned the tools.

“This file is named but its content is not there.”

The round-2 re-audit has now failed three times, and not once on its own merits. Attempt 1, Qwen3.8-Max: repeated provider-side failures before grading began. Attempt 2, Kimi K3: it received the Constitution as a PDF, could see the filename and not the contents, said so, and stopped. Attempt 3, Qwen again: the same defect. **All three refusals were correct behaviour.** The common factor across two different providers on two different days was the PDF, so the Constitution now ships to auditors as text — a pdftotext -layout extraction carrying the source PDF's SHA-256 in its own header and verified byte-for-byte against a fresh extraction. Earned rule 28 applied: a format the recipient cannot open is not fixed by asking the recipient to try harder. **One thing recorded against Claude.** The first diagnosis was built from a truncated sentence visible in a screenshot — “this file is named but its content is” — and asserted as a parsing failure. The full reasoning, recovered later, said something stronger and more specific. The fix was right; the confidence behind it was not earned at the time it was stated.

“An audit that produced no report still produced eleven defects.”

Attempt 3 could not read the standard and correctly refused to grade 44 rules it had not seen. It then ran the instruction's own Section 7 checks — which are defined in the instruction rather than in the Constitution — and returned eleven observations. Its reasoning is preserved at `qwen_reasoning_1.txt` through `_4.txt` in the repository. **Every one was treated as a CLAIM and checked against source before a line of code was written.** None evaporated. Two got worse under checking. That discipline is the entry: a reviewer's finding is a hypothesis with a file and a line attached, and the difference between an audit and a rumour is whether someone opened the file. The eleven span fabricated levels, false claims about files, a run identity that did not identify the run, a smoothing mismatch between an indicator and its own fallback, an unset HTTP timeout, and a directory created as a side effect of an import.

“Two different trading plans on the same candles, recorded as the same run.”

`core/decision_log.py` has carried this sentence since the Finding 6 fix was written: the audit's required action asks for “all decision-affecting configuration, **including risk-model multipliers and bias weights.**” Fifteen lines below it, the fingerprint dictionary named one module — the bias engine. The weights were covered. The risk multipliers were not. Changing `ATR_STOP_MULT` from 1.2 to 1.5 moves the stop and all three targets on every run, and left the hash, the config snapshot, the module snapshot and provenance byte-identical. Item 6 is Traceability, raised to Critical by Viktor on 29 August — broken inside the fix written to satisfy it. **It was not a name missing from a list.** The multipliers were instance attributes on `RiskModel`, and the snapshot mechanism reads module attributes; there was nothing for the list to name. Viktor ruled the constants move to module level rather than the mechanism grow to reach inside objects — the smaller change, and the shape the one compliant module already had. `__init__` is gone entirely, so no instance state can drift from what the record reports. Proven behaviour-preserving across 1,728 stop and target combinations and 192 regime combinations: zero differences.

#53

A Test That Described the Defect It Could Not Catch

September 3, 2026

PROCESS — RECORDED

“...so the default never fires and the panel prints 'saved to None'.”

That sentence has been the failure message of a test in `test_traceability.py` since sequence item 12. It is correct. It describes, exactly, a defect that was live in the engine the whole time — and the assertion beneath it only checked that the OLD hardcoded default was gone from `panel_render.py`. The `None` arrived from somewhere else: the router built the decision object with `str(chart_path)`, and `str(None)` is the four-character string “None”, which is truthy, which passes the panel's own gate. The panel printed **Chart saved to None** — a file that does not exist — for the life of the engine. **A second test had the same gap from the other direction.** `test_unwritable_log_dir.py` already pointed the chart directory inside an unwritable path, so every run of that file drove the engine through this exact state, and it asserted only the log path and the archive. Nobody looked at the chart. A test whose failure message documents a defect it cannot detect is worse than no test: to anyone auditing the suite it reads as coverage. The claim is made in one file and the value built in another, so the check now follows the value.

#54

The Door Closed and the Window Left Open

September 3, 2026

DECISION — ADOPTED

“Twelve of twelve points for a volume node that was never located.”

`StructureEngine.analyze()` wrapped each of five sub-routines in its own handler and substituted a value on failure — the previous bar's regime, “NONE”, “NEUTRAL VOLUME”, and for both volume nodes and the swing level, **the current price**. None of the five recorded anything, and the function had no channel to record through, so a crashed detector produced an ordinary-looking reading and the run was never marked degraded. **The volume-node case is Finding 3 again.** Entry quality scores structure proximity as $\text{abs}(\text{close} - \text{hvn}) / \text{close}$, which with `hvn` equal to `close` is exactly zero — inside the < 0.015 band, awarding the full 12 of 12. The comment describing that arithmetic has sat in `entry_model.py` since 1 September, written when Finding 3 was fixed. That fix changed the *consumer's* fallback to `NaN` and left the *producer* handing down a finite number equal to the price. It even cites rule 18 — fixed the instance it was looking at and left its twin. This is the twin of that twin. No handler invents a measurement now: an unlocated level is `NaN`, which every consumer already guards, an undetermined label says UNKNOWN rather than borrowing NEUTRAL, and every failure reaches the run's degradation list.

#55

Found by Reading the Panel, Twice in One Morning

September 3, 2026

FINDINGS — FROM EXECUTION

“226 tests, and neither defect was found by any of them.”

The first was Claude's, shipped that hour. The patch above replaced a panel line that ended in a newline with a computed one that did not, and the engine printed `SWING STRUCT : $0.4700 (Lookback 8)STOP LOSS : $0.4636`. Thirteen new tests, a negative control, a full-suite run and a golden-snapshot check all passed — because every assertion asked whether a substring was present, and it was. Nothing in the suite rendered the panel and looked at its shape. Viktor found it in one live run, which is earned rule 30. **The second was not Claude's.** The same panel printed `MACRO TREND: BULLISH` and, four lines below, “The higher timeframe is neutral.” The branch that produced it fired when the BIAS was neutral and then made a claim about the MACRO — false on every neutral-bias run, which is a common state and was the state that morning. Same shape as the contradiction that exposed the direction Critical the day before. The logic is now a named function with a fourth branch, extracted so the four cases can be tested at all: inline, the false branch could only be reached with market data that happened to produce a neutral bias under a directional macro, which is why no fixture held it and no test could have.

#56

A Conversation Is Not a Record

September 3, 2026

PROCESS — RECORDED

“A full day of findings, rulings and sizings existed in a chat window and nowhere else.”

Late in the session Viktor asked whether Claude would still have everything the following day. It would not have. Eleven verified findings with their file and line, four patches, three rulings made, three owed, sizes for the remaining work and the pre-audit checklist were all established in one conversation and written into no document. It was caught because he asked — which is precisely the save-by-vigilance earned rule 28 exists to replace. **The fix is a handover check, run by Claude unprompted at the end of every session**, asking six questions: is the state in the entry-point document rather than only in chat; are the day's rulings recorded; does `git status` show untracked files that matter; are these notes current or the gap stated; are loose patch files unapplied; is any evidence still in a chat window. On its first run it found three files left on the floor, one of them sitting there since the previous day. **The same conversation split the project's goal in two.** Phase 7 is now both the technical portfolio project in Viktor's 2026–2028 career plan and an engine he intends to finish, and those end months apart. Portfolio-ready is reached, tagged, and only then does backtesting begin — on top of the tag, so that the phase which destroyed the previous build cannot damage the version he submits. Earned rules 30 through 34 were written the same day.

#57 The Trend the Bias Engine Could Not See
September 4–5, 2026

DECISION — ADOPTED

“Continuation was zero, so the trend was flat — whatever the trend was doing.”

Ruling 3, and patch I. `bias_engine` was never told which way the trend pointed. It inferred direction from the sign of `continuation_strength`, so whenever continuation came out at zero the engine read the trend as flat — silently, and regardless of what the trend actually was. `indicators/trend_health.py` had computed the direction all along and thrown it away. It is now exposed as `trend_direction` and `trend_direction_sign`, and the sign is a **required** parameter of the bias engine, validated against (-1, 0, 1) and raising rather than defaulting — an inferred value replaced by a declared one. **The order in which this was decided is the entry.** How often the engine silently flattened a real trend is a question with an answer in the data, and the answer was obtained across 9,800 live bars *before* the ruling was made, not afterwards to justify it. Earned rule 36 was written from this: a ruling made first and measured second is a hypothesis with a decision attached to it.

#58 A Magnitude Offered as a Direction, and a Lean Too Small to Act On
September 5, 2026

DECISION — ADOPTED

“A magnitude was being cited as evidence for a direction it cannot indicate.”

Rulings 1 and 2, both in patch K. **Ruling 1:** the reason strings the engine writes for a BULLISH or BEARISH call cited trend health as supporting evidence. Trend health is a magnitude; it says how strong, never which way. Sitting in a directional sentence it read as directional support that it was not. It now prints as `trend strength 78/100 (rising)`, with the direction word coming from the sign patch I had just exposed — ruling 1 was only buildable because ruling 3 landed first. **A correction to the record:** Claude had claimed the panel's TREND line carried the same defect. It does not. It prints two true facts side by side and asserts no relationship between them; only the reason strings made the claim. **Ruling 2:** an ACTION could be issued off a bias that barely leaned at all. `MIN_ACTION_BIAS = 30.0` now floors it, and is fingerprinted so it enters the run's identity. Claude first argued for folding it into the existing `RAW_BIAS_THRESHOLD` and reversed that while building it: one asks *does the blend lean far enough to call a side*, the other asks *is that lean strong enough to act on*, and one number tuned for both makes one of the two answers an accident of the other. Seven fixtures that built a bias with no score at all were **completed** rather than worked around — relaxing a floor to accommodate incomplete test data is how a floor stops meaning anything. Patch J, alongside, extracted `volume_agreement` into a module-level function for the same reason patch D extracted `macro_agreement`: logic reachable only through a full run cannot be tested in isolation.

#59

Three Patches Built on a Base That Had Already Moved

September 5, 2026

PROCESS — RECORDED

"The staged copy is a snapshot, not a view."

Patch J was generated against an `engine_core.py` staged before patch I landed, and failed 39 tests. Patch K was rebuilt twice against a `decision_log.py` missing an anchor added hours earlier. None of the three patches was wrong; the base each was diffed against was. **The failure mode is asymmetric, which is why it earned a rule.** A patch built on a stale base fails loudly if you are lucky and applies cleanly if you are not — and the clean application is the one that reaches the repository. Earned rule 35: re-stage every base file immediately before generating a diff. Recorded alongside it, because it is the same week and the same shape: a test written in this batch asserted on the substring "support" against a note that quotes a label reading BULLISH VOLUME SUPPORT. Earned rule 30 had been written three days earlier about exactly that, and was violated in a test written to guard its cousin. Rule 37 came from this: knowing a rule and applying it are different acts, and the rules that repeat are candidates for a mechanical check rather than a firmer intention.

#60

Explaining in the Right Place

September 5, 2026

DECISION — ADOPTED

"A long thread pays for its own history on every turn."

Viktor asked Claude to decide and implement how to reduce token usage, on one explicit condition: it must not weaken the engineering. Two changes, decided by Claude and accepted. **First, when a patch ships with a commit message, the chat reply carries only the predictions to check and the commands to run.** What the fix does, why, the weights, the verification and the mistakes made along the way are already in the message he is about to read; saying it twice is pure waste, and the chat copy is the one that does not survive the session. **Second, a thread is closed and a fresh one started once a piece of work has finished** — "Continue Phase 7" is enough to resume, and the handover check of Entry #56 is what makes that safe rather than reckless. What was explicitly **not** cut: negative controls, golden checks, equivalence runs, and naming every consequence of a change before it is made. Those are the reason the patches have held, and they were the condition. This must not become a reason to explain less. It is a reason to explain where the explanation survives.

“Nothing could reach it — which is exactly why two audit passes left it there.”

Patch L, finding 6. Two fabrication constants survived in `engine_core.py` after item 9a removed their siblings from `indicators.py` and Finding 3 removed them from `entry_model.py`: a substitute trend dict `{"trend_health": 50.0, ...}`, and `atr_val = ... else current_price * 0.02`. Both were unreachable. `compute_trend_health` is total, so nothing could reach the first handler; section 2 halts when ATR is absent, so nothing could reach the second. **The trend one was worse than a fabrication, and only the negative control showed it.** Its substitute dict has no `trend_direction_sign`, which the next stage reads by subscript *outside* the try. Run against pre-fix code, a broken trend contract does not degrade the run — it dies with `KeyError: 'trend_direction_sign'`, reported as neither a trend failure nor a bias failure. Found by running the test, not by reading the code.

“A test had asserted, on that claim, that these paths cost nothing.”

Patch L, finding 5. The RSI and ATR fallbacks smoothed with a simple moving average; `pandas_ta` smooths both with Wilder's RMA. So the sentence in `add_technical_indicators`' own docstring — “recomputes the same quantity by another route” — was false, and it was the stated grounds on which `test_degraded_state` asserts these paths are **not** degradations. On the pinned fixture, final bar: RSI 84.45 against `pandas_ta`'s 69.14, and an ATR 1.80% low. That test had been passing on a false premise for as long as it existed. **The choice was Claude's, and was made by reading the repository rather than by preference.** Viktor: *“None needs a ruling from me.”* Match the smoothing, or record which path ran? Recording the path leaves `test_degraded_state`'s assertion false and adds a flag to explain why. Matching makes the assertion true. The repository already contained the argument; the job was to find it, not to have an opinion.

““2 errors” — and the result for the other 251 tests was unobtainable.”

Both of these were found because the verification step ran, not because anyone was looking for them. **The `pandas_ta`-free control could not run at all.** `test_macro_agreement` and `test_volume_agreement` import a pure function whose module chain reaches `pandas_ta`, so both errored at collection; pytest reported two errors and executed nothing. That had been silently true since those files were written, and the degraded-dependency result nobody could obtain was the one the fallback work depended on. `pytest.importorskip` makes it the skip it should always have been — 0 errors, 159 passed, 88 skipped. **And a module-level object defeated a deliberate lazy import.** `SignalRouter.__init__` imports `engine_core` inside the function, which is what lets `live_trading` be imported without `pandas_ta`. A module-scope `LiveTradingSimulator()` called that constructor at import time and undid it. Verified in both directions rather than argued: pre-fix, `import live_trading` raises `ModuleNotFoundError`; post-fix it succeeds.

#64 Two Series Paired by Position, Correct by Accident
September 5, 2026

DECISION — ADOPTED

"The printed label changed in 4 of 31 positions, and the evidence line gave no tell."

Patch O, finding (a). Correlation and beta between AERO and BTC were computed by pairing the two series **by position** after both timestamp indexes had been discarded. In the ordinary case both fetches return the same 450 candles, so positional pairing is timestamp pairing and the code is right by accident — which is why it survived, and why the golden snapshot did not move when it was fixed. On the pinned fixtures the two indexes share all 450 timestamps and old and new agree to the last decimal. **It stops being harmless the moment the series differ by one bar:** a candle closing between two sequential API calls, an exchange gap, a stale feed. Measured on the fixtures by dropping one BTC bar from inside the window, at all 31 positions: correlation moved by a median of 0.105, beta by 0.135, and the printed label changed in 4 of the 31. `n_observations` read 30 either way, so the panel's own evidence line could not reveal it. The join is now on the timestamp index, and the observation count is the count of the aligned pairs.

#65 A Zone Whose Lower Bound Was Above Its Upper, and a Price Nobody Read
September 5, 2026

FINDINGS — FROM EXECUTION

"ENTRY_ZONE : \$0.4981 - \$0.4918"

Patch P closed finding (4) and a display defect that was not on any list. **The display defect was found by reading the panel.** `engine_core` assigned `EMA_20` to `zone_lower` and `EMA_50` to `zone_upper` unconditionally, so in an uptrend they printed inverted. `entry_model` swaps them before scoring, so the arithmetic was correct the whole time and only the display was wrong — which is why no test could have caught it. Every test asserted on the numbers. **Finding (4) was the familiar shape:** `close * 0.99 / close * 1.01` for a missing EMA pair, reachable only when an EMA is missing, which a healthy run never is. Five fabricated constants across two files, and the worst was not on the list — a close price that could not be read became `$1.00`, so a run with no data at all reported ACTIVE ENTRY_ZONE, 30 of 30, against a price nobody had read. **Three of this project's defects have now been found by running the engine and looking at the output.** It remains the cheapest detector the project has. Left open and written down rather than assumed: whether any other reader of the zone is missing the same swap.

#66

Five Components Summing to 39, Under a Total of 45.18

September 5, 2026

DECISION — ADOPTED

“Not wrong, unexplained — and for a number an operator acts on, that is its own defect.”

Patch Q. The entry-quality panel had been printing a total its own component list did not add up to, recorded on 2 September as “not wrong, unexplained”. An operator meant to act on that number either works the gap out themselves or stops trusting the number, and both are failures. Three causes, none visible anywhere: sub-scores rounded for display while the total was not, three confluence multipliers applied after the sum, and a clip at 100.

And a fourth thing the clip was hiding. The five components add to **102, not 100**, while the docstring directly above the list said 100. With full confluence a perfect setup reaches 118.08 and loses the difference silently — the clip was concealing an arithmetic error in the scoring definition itself, not merely bounding an output. The panel now prints the subtotal out of 102, the multiplier broken into its three factors, and a Clipped line when the clip actually fires. The column adds up.

#67

One Fact Declared in Four Places

September 5, 2026

OBSERVATION — FILED FOR
THE RECORD

“Nine new fields vanished between the engine and the panel, with no error and no failing test.”

Recorded, not fixed. `signal_router.py` rebuilds the entry block **field by field**. Anything the engine adds that its list does not name is dropped before the panel or the decision log ever sees it — silently. Patch Q's nine new reconciliation fields disappeared exactly that way on the first attempt: the lines simply did not appear, and nothing failed. The entry block's shape is now declared in four places — `entry_model`'s return, `engine_core`'s dict, `signal_router`'s rebuild, and `decision_contract`'s TypedDict — four copies of one fact, which is the defect class this project has recorded more often than any other. **Restructuring it is larger than any single finding warranted, so it is written down here instead of being done quietly at the end of a patch about something else.** Earned rule 24 in a different costume: the fix that belongs in its own commit does not get smuggled into one that has already made its claim.

#68 **Four for Four: the Fabrications That Survive Audits Are the Unreachable Ones**

September 5, 2026

OBSERVATION — FILED FOR
THE RECORD

“Nothing tests a path a healthy run never takes, and nothing else notices it is wrong.”

Item 9a, Finding 3, Finding 6 and Finding 4 were four separate findings across three weeks, and all four removed fabricated constants from code paths that a healthy run never reaches. That is not a coincidence and it is now the strongest generalisation this project has produced. **A constant on a live path is checked by every run and every fixture. A constant on a dead path is checked by nothing** — not by the suite, which cannot enter the state, and not by a reader, who sees a defensive default and moves on. So it survives audit after audit while looking exactly like prudence. Unreachable is not safe: it is one edit to an invariant elsewhere from becoming the live path, and on the day that happens the engine will print a fabricated number with complete confidence and nothing in the record will say where it came from. Finding 6 proved the sharper version — its dead branch would not merely have fabricated, it would have crashed the run with a `KeyError` attributed to neither stage. **The generalisation worth carrying to other projects:** when auditing for invented values, search the unreachable branches first, not last.

#69 **A Test Suite That Could Reach the Internet**

September 5, 2026

OBSERVATION — FILED FOR
THE RECORD

“Two rejected `CONNECT`s to an exchange, from a run in which every route was pointed at a dead port.”

During a full-suite run the sandbox's egress proxy logged two rejected `CONNECT` attempts to `api.mexc.com:443`. Every test that routes or fetches sets `base_url` to a dead local port first, and each of them was checked; the source of the two attempts was not identified. **Recorded rather than chased, and recorded rather than dismissed.** The attempts were blocked, so nothing reached the network and no result depended on what an exchange returned — this run. A suite that *can* reach the internet is a suite whose result depends on the network on some other run, on a machine with no proxy in front of it, and the failure would arrive as an unexplained flake rather than as an error naming its cause. It is written here so that the next unexplained flake has somewhere to start, which is what this log is for.

“Every queued fix has landed. That is not the same as a clean report.”

Between 3 and 5 September the suite went from 226 tests to **319 passing**, and the “What is left to fix” table in PHASE7_NEXT.md reached zero rows — three rulings built, and all seven queued findings closed, across patches I, J, K, L, M, O, P and Q. **The release gate stays shut, and the distinction is the entry.** Unresolved in this project means fixed *and* re-audited; every one of these fixes was written by the party under audit, and a fix nobody independent has seen is a claim, not a verdict. What stands between here and the re-audit is the five-item pre-audit checklist, and the first two items on it are the two that were skipped last round: rebuild the package against current code so the auditor grades what exists, and commit everything before building it, so the packaged files match a commit rather than a working tree. **A completed audit is not an open gate either.** If the report returns a Critical, the gate stays shut and the cycle repeats. The milestone is a clean report, not a finished one.

"Three Qwen calls, then Kimi. The record has the last two attempts the wrong way round."

Earned rule 29 says to reconstruct a factual record from the provider's billing log rather than from memory, because every provider bills per request and the bill cannot be mistaken about what was called. It was written on 2 September after the independence ledger turned out to have been written from recollection. Applied a second time, to the question of which model ran which attempt at round 2, it found three things. **One: the order is wrong.** The log for 2 September reads Qwen 10:53, Qwen 15:38, Qwen 15:42, Kimi K3 15:58. Entry #50 records attempt 2 as Kimi and attempt 3 as Qwen; Kimi ran last, after all three Qwen calls, so the eleven observations of Entry #51 came from a Qwen run that preceded it. Viktor's recollection — "we ran Qwen three times" — was exactly right. **Two: Claude's own reasoning was wrong, and in a way worth naming.** Having found that those transcripts cite a superseded revision of the reviewer instruction, Claude offered the files' modification times as evidence that the correct package existed before the run and was not used. A modification time records when a transcript was saved, not when a model ran. Ordering of writes was presented as ordering of events; the claim was withdrawn. **What the clock does establish, once both sets of times sit on one axis, is stronger:** the UPLOAD_THESE folder was written at 16:47:56 and every one of the four calls predates it. **No attempt at this round has ever received it.** Nobody uploaded the wrong folder — the folder did not yet exist, and the eleven observations came from a run holding rev 2 and a file named for the Constitution whose contents were the engine source bundle. Two distinct payloads confirm the shape: 213,581 input tokens in the morning, then 196,283 for all three afternoon calls across two providers — one upload, retried. **Three: a model named in this log was never called.** Entry #22 records Run 1 as a blind review by DeepSeek V4 Pro. The bill says DeepSeek V4 **Flash 0731**. Independence is tracked at the laboratory, so the ledger's verdict is unchanged and DeepSeek is spent either way — but the record names a checkpoint that was never billed, which is the same error rule 29 was written about. **Left open rather than resolved:** the log shows nine Luna Pro calls between 27 August and 3 September, considerably more than this document's account of Luna Pro accounts for, and three of them on 3 September map to nothing recorded anywhere. Kimi K3's 2 September call returned 36,085 output tokens, which is not the truncated stub the record describes when it leaves Kimi's spent status undecided. Both are noted here so the next reader starts from the discrepancy rather than rediscovering it. **The generalisation, now paid for twice:** where a project keeps a record of what happened, ask whether some system already recorded it as a side effect of doing its own job, and prefer that system. This project's account of one failure was built first from a truncated screenshot, then from file timestamps, and only the bill settled it.

“Seven of the forty-four rules carry their previous verdict in a code comment.”

The round-3 package was built and then **searched** before it was sent — the step that the previous two rounds did not take. Section 2 of the reviewer instruction told the auditor, in as many words, to stop if it found a named prior auditor, a count of Compliant and Non-compliant items, or a list of Critical findings *in either of the two code bundles*. The bundles contain all three. Items 3 and 6 described as rated Critical; Item 18 as kept Compliant; Item 16 as having gone Non-compliant; Tier 3 items 3 and 4 as currently Non-compliant; Tier 4 item 2 as rated Compliant with the previous auditor's reasoning quoted. Three separate counts of Criticals. Luna Pro and GLM named. **A reviewer obeying the instruction would have refused to grade, and that would have been the fourth attempt at this round to end without a report and the third in a row killed by the package rather than by the code.** The contradiction had been half-seen already: Rev 3 records scoping this same stop condition to resolve a clash with Section 9, and that scoping drew the line at *the instruction may tell you counts, the artifact must not tell you content*. Nobody then checked whether the artifact told content. **Viktor's ruling:** scope the condition to the Constitution file, count and disclose exactly what the bundles hold, and ask the reviewer to grade all forty-four and say in Part 6 — “I don't know” included — whether a comment moved it. The comments are not stripped: Section 4's argument stands, that removing them is the audited party editing its own evidence and destroying what Item 8 exists to test. **And separately, the structural fix, deferred on purpose:** the audit narrative moves out of code comments and into this document, but not inside the audit it would affect — doing it now would hand the auditor a codebase edited to look better for its grader. **The reason this matters more than a fixed contradiction:** a reviewer that stops tells us loudly. The failure one step over is silent — a reviewer that does not stop, reads “Item 18, kept Compliant”, and grades Item 18 Compliant. Nothing in its report would reveal that, and the report would look entirely ordinary. That is earned rule 28's test applied to the audit process itself, and it is why the same instruction now asks the reviewer to search its own package before grading.

Document History

Version	Date	Notes
v1.0	August 25, 2026	Converted from the standalone Engineering Note #1 into this standing log, at Viktor's request. Entries 1–3 carried over unchanged, split from one combined note into three individually tagged entries.
v1.1	August 26, 2026	Entries 4 through 9 added, tracking the Constitution's Rev 3 through Rev 5 and the new Tier0 Companion document. No earlier entry rewritten — Entry #9 corrects scope drift introduced in Entry #8's phrasing without altering Entry #8 itself, per this document's own no-silent-edits rule.
v1.2	August 26, 2026	Entry #10 added, closing Entries #1 and #6 — the credential gap is adopted into Constitution Rev 6 as Tier 1 Items 18 through 21, with operational detail in the new Phase7_Credential_Security_Protocol.pdf, and the audit now has a named independent auditor. New DECISION — ADOPTED status tag introduced for entries that record a decision actually made rather than a candidate awaiting one. Entries #1 and #6 left unedited; Entry #10 is where their closure is recorded.
v1.3	August 26, 2026	Entry #11 added: Viktor named Grok as the independent auditor for Steps 3, 4, and 8, once released, recorded in Constitution Rev 8. Answers the authorship-vs-judgment-independence question Rev 7 had just sharpened via Reviewer 4's identity.
v1.4	August 26, 2026	Entry #12 added, and format changed to hold it: a new gold-framed highlighted_entry_box treatment, used only when Viktor asks for an entry to be marked especially important. Documented in "How This Document Works" as a rare, deliberately non-standard marker rather than a new status tag. First use: a reflection on the audit loop's design, and the two places its soundness still depends on discipline rather than mechanism.
v1.5	August 26, 2026	Entry #13 added: criteria for evaluating any future auditor substitution — independence as a gate, power and cost as the actual trade-off, cost mattering because Step 8 makes the audit a recurring expense rather than a one-time one. Extends Entry #11 without editing it; Grok remains the named plan.
v1.6	August 26, 2026	Entry #14 added: Viktor ratified Constitution Rev. 8. Recorded in the Constitution's own Version History as a new RATIFIED row, not a Revision 9 — no rule text changed, only status. Scope freeze now in force for real; Step 2a of the audit sequence may begin.
v1.7	August 26, 2026	Entry #15 added: Gemini's assessment of the finished, ratified Constitution, logged and gold-highlighted at Viktor's request — second use of the highlighted_entry_box treatment, still rare. Not a structured finding and not the audit; flagged as such in the entry itself.
v1.8	August 26, 2026	Entry #16 added, recording how Entry #15's assessment was obtained — solicited by the party being evaluated — and the critical counter-review Viktor then commissioned. Entry #15 left unedited. Deliberately not gold-framed: the highlight marks Viktor's request for emphasis, not epistemic weight, and a counterweight entry that competed for attention would defeat its own point.
v1.9	August 26, 2026	Entry #17 added: Step 2a complete. Claude assembled the audit package from the actual working repository (19 source files, 4 evidence files not present in the public repo) at Viktor's instruction. Zero findings written, per Step 2a's own rule. Notes that Entry #3's file list is now superseded by the package's manifest, without editing Entry #3 itself.

Version	Date	Notes
v1.10	August 26, 2026	Entry #18 added: before running Step 3, Viktor noted Grok had already seen the Constitution in an earlier, separate conversation. Named as an independence risk on the same grounds as Entry #16, resolved by running Step 3 in a fresh conversation with no shared memory of the earlier one. The public GitHub repository was considered and deliberately left published.
v1.11	August 27, 2026	Entries #19, #20 and #21 added. #19 logs Grok's review of the Constitution as an external assessment, not Step 3, with its factual claims verified and its weaknesses named. #20 records Step 3 being paused and the auditor plan changing from Grok to a panel of models with no prior involvement, run pay-per-use; no revision of the Constitution was needed, since Rev. 8 named Grok as a plan rather than a requirement. #21 records an internal contradiction found in Claude's own drafting of the Minimum Viable Audit gate, deliberately recorded rather than repaired. Two new Constitution Version History rows (AUDITOR, DEFECT); no rule text touched and the register still frozen at 21 / 7 / 10 / 6.
v1.12	August 27, 2026	Entries #22, #23 and #24 added, covering the audit actually being run. #22 records Run 1, the blind source-only review by DeepSeek V4 Pro, whose ten findings were each checked against the real source and held. #23 records the first Step 3 attempt truncating inside its own reasoning at a default token ceiling, and the resulting split of the register across three scoped runs; the corresponding wording change is recorded as a new Section 6 in Phase7_Audit_Execution_Instructions.pdf rather than made silently. #24 records Run A, the Minimum Viable Audit gate: Items 2 and 18 Compliant, Items 3 and 6 Non-compliant, the Entry #21 DEFECT contradiction resolved by the auditor in favour of the four-item gate, and a false claim found in Claude's own Step 2a package. #24 also records, and withdraws, a false accusation Claude made against the auditor — a citation Claude called invented turned out to be accurate and located in the other evidence file, which Claude had not searched. Two genuine disagreements between Claude and the auditor are recorded unresolved and go to Viktor. No rule changed; register still frozen at 21 / 7 / 10 / 6, and the freeze stays in force until every Tier 1 item has a finding.
v1.13	August 27, 2026	Entries #25 and #26 added. #25 records Run B — the seventeen Tier 1 invariants outside the gate — with every substantive claim verified against the source, two precision errors in the auditor's evidence noted, two Compliant ratings disputed by Claude over the indicator cache, and the observation that two material Item 14 defects appear in Kimi's truncated first attempt but in neither run that completed. #26 records the scope freeze lifting: all 21 Tier 1 items now have findings, meeting the mechanical trigger Reviewer 4 forced into Next Steps at Revision 7. A new AUDITED row was added to the Constitution's Version History recording the same fact; no rule text was touched and the register stands at 21 / 7 / 10 / 6, no longer frozen. Four adjudications remain open for Viktor.

Version	Date	Notes
v1.14	August 27, 2026	Entry #27 added, recording Run C — Tiers 2, 3 and 4 — which completes the register: all 44 rules now carry a finding. Twenty-three items graded; four Tier 2 Non-compliances including a module that rewrites its caller's DataFrame in place, and a dependency manifest that omits two packages the code imports while declaring one nothing imports. Recorded as a method difference: Run C had web access and fetched the public repository, which Runs 1, A and B did not. Claude repeated that fetch independently and closed two Unknowns — no tags or releases (T3-7 becomes Non-compliant) and twenty-three commits on master (T3-9's depth question resolved). One contradiction between Run B and Run C is recorded with Run B's reading preferred, on source evidence.
v1.15	August 27, 2026	Entry #28 added, recording a hostile external review of the Constitution commissioned by Viktor from an uninvolved model. Two of its findings adopted into the Constitution the same day — a release gate, since the freeze-lifting definition established that the audit ran but attached no consequence to what it found; and amendment control, since the now-live amendment process would permit a Tier 1 invariant to be weakened by a short note reviewed only by Claude and Viktor. A front-matter statement of what the engine is was added alongside them. A third finding, Item 20 not covering crash-reporter capture of the process environment, is recorded and left unfixed: it requires amending a Tier 1 invariant, which the newly adopted rule says Claude may not do without external review. The review's remaining nineteen proposed invariants are declined as calibrated to a production trading system this engine is forbidden to become, and go to Future Amendment Candidates. Register unchanged at 21 / 7 / 10 / 6.
v1.16	August 29, 2026	Entries #29 through #33 added, covering the work between the audit closing and the remediation sequence arriving. #29 records Phase A — eighteen tests built and run, five passing, thirteen failing as written acceptance criteria, and the golden-path test that was shipped unverified and turned out to be wrong. #30 records the three defects that first run found, which four audit passes across three models had missed: a severity escalation on the clone failure, and two findings nobody had. #31 records the model-independence table in one place for the first time, and the rule that independence is tracked at the lab rather than the checkpoint. #32 records machine learning put on ICE with five written conditions for revisiting. #33 records Step 5 running on GLM 5.3, nine of nine source claims verifying, and Claude coming one sentence from a third false assertion of absence. Register unchanged at 21 / 7 / 10 / 6; the adjudications Step 5 asks for remain open.

Version	Date	Notes
v1.17	September 2, 2026	Entries #34 through #42 added, covering the six days between the audit closing and this row — a period this log had not recorded at all. #34 the machine rebuild and the golden baseline surviving it across a numpy major version. #35 Step 8, the independent re-audit: five Criticals, ten further findings, release gate not met. #36 remediation batches 1 and 2, including 46 tests that returned where they should have skipped. #37 items 3, 11 and 14 delegated by Viktor and ruled. #38 the macro degradation, dependency pinning, the four unguarded tests and two stale docstring labels. #39 Finding 3 — a Critical that had never been entered into any roadmap, found by reading the audit report rather than the summary derived from it, with a defect class wider than the two instances the report named. #40 the status sweep: three findings already closed by commits that never named them, and a count wrong by three. #41 the Step 8 auditor not being on the clean list, recorded rather than quietly noted. #42 the full document set read end to end, correcting four things no working document held. Register unchanged at 21 / 7 / 10 / 6, no longer frozen. Two amendments owed, neither adopted; the release gate stands closed on Item 6.
v1.18	September 2, 2026	Entries #43 through #49 added, covering the day the last Critical closed and a new one opened. #43 Findings 6 and 7 — input hashing, a raw-candle archive pruned at ninety days on Viktor's ruling, and the Item 6 lineage chain persisted. #44 a defect that shipped: a path separator that made the golden snapshot match only on the platform it was baselined on, caught on Viktor's machine on the first run. #45 the direction-source Critical — a CONSERVATIVE LONG printed over a stop above price and three descending targets, found by running the engine on live data, and missed by an independent 44-rule audit plus four passes across three models. #46 an unwritable log directory destroying a whole run, with Claude recommending refusal and being overruled. #47 a generated fixture that lets the suite reach the state that hid #45. #48 the session / training / lineage distinction ruled, and the independence ledger corrected from the billing log — Mistral had been wrongly listed clean for a week. #49 the round-2 audit package generated rather than assembled, and eight tools left enabled on the first attempt at it. Register unchanged at 21 / 7 / 10 / 6. Every Critical the Step 8 audit raised now has a fix that has landed; the release gate stays shut until a re-audit has seen them.
v1.19	September 3, 2026	Entries #50 through #56 added. #50 the re-audit's third failed attempt and the Constitution shipped as text, because two providers on two days could see its filename and not its contents. #51 eleven defects returned by an audit that produced no report, every one checked against source before it was believed. #52 the run hash that did not cover the risk multipliers — Item 6 broken inside the fix written to satisfy it. #53 a test whose failure message described a live defect it could not detect. #54 the volume-node fabrication reached through the producer after Finding 3 closed it at the consumer. #55 two defects found by reading the panel in one morning, one of them shipped by Claude an hour earlier and passed by all 226 tests. #56 the handover check, and the project goal split into a tagged portfolio milestone and a later finished engine. Suite 196 to 226. Register unchanged at 21 / 7 / 10 / 6. Earned rules 30 through 34 written the same day.

Version	Date	Notes
v1.20	September 5, 2026	Entries #57 through #70 added, covering 4–5 September — the three open rulings built and the whole of the “What is left to fix” table closed, across eight patches. #57 ruling 3, the bias engine inferring trend direction from the sign of continuation strength and reading a real trend as flat, measured across 9,800 live bars before the ruling rather than after. #58 rulings 1 and 2 — trend health, a magnitude, offered in reason strings as directional support, and a minimum bias strength kept as its own fingerprinted constant rather than folded into an existing threshold. #59 three patches diffed against a base that had already moved, and earned rule 30 violated in a test written to guard its own class. #60 the usage discipline Viktor delegated: reasoning goes where it survives, verification is not cut. #61 two fabrication constants that survived two passes because they were unreachable, one of which would have killed the run rather than degraded it. #62 a fallback whose docstring claimed an equivalence it did not have, and the test that had asserted on that claim for as long as it existed. #63 a degraded-dependency control that reported two errors and executed nothing, and a module-level object that defeated a deliberate lazy import. #64 correlation and beta paired by position after both timestamp indexes were discarded — correct by accident, and the printed label moving in 4 of 31 measured positions once the series differ by one bar. #65 an entry zone printed with its lower bound above its upper, found by reading the panel, and a close price that could not be read becoming \$1.00. #66 five entry sub-scores summing to 39 under a printed total of 45.18, and the clip that was hiding a component list adding to 102 against a docstring saying 100. #67 the entry block’s shape declared in four places, recorded rather than restructured inside a patch about something else. #68 the four-for-four generalisation: the fabrications that survive audits are the ones on paths a healthy run never takes. #69 two rejected CONNECTs to an exchange from a suite in which every route was pointed at a dead port, source unidentified, recorded rather than chased. #70 suite 226 to 319 passing and the fix table empty, with the release gate still shut because unresolved means fixed and re-audited. Register unchanged at 21 / 7 / 10 / 6. Earned rules 35, 36 and 37 written across the same two days.
v1.21	September 5, 2026	Entries #71 and #72 added, both corrections to this project’s own record rather than to its code. #71 applies earned rule 29 a second time: the provider’s billing log shows three Qwen calls followed by Kimi K3 on 2 September, so Entry #50 has the last two attempts of round 2 in the wrong order and the eleven observations of Entry #51 came from a Qwen run that preceded Kimi rather than followed it. The same log establishes that no attempt at round 2 ever received the UPLOAD_THESE folder — it was written after the last of the four calls — and corrects Entry #22, which names a DeepSeek checkpoint the bill says was never called. Claude’s own inference from file modification times is withdrawn in the entry. Luna Pro’s call count and Kimi K3’s spent status are recorded as open questions, not resolved. #72 records a defect found by searching the round-3 package before sending it: Section 2 of the reviewer instruction told the auditor to stop if the code bundles contained prior-audit outcomes, and they contain a stated verdict for seven of the forty-four rules, three counts of Criticals, and two named prior reviewers. Viktor ruled the condition scoped to the Constitution file and the bundles’ contents counted and disclosed, with the audit narrative to move out of code comments into this document later — not inside the audit it would affect. Register unchanged at 21 / 7 / 10 / 6. The release gate remains shut.

Future entries get appended below this line, in order, each with its own number, date, and status tag — this document grows with the project rather than being rebuilt each time.